

Inteligență Artificială

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

Radu Ionescu

raducu.ionescu@gmail.com

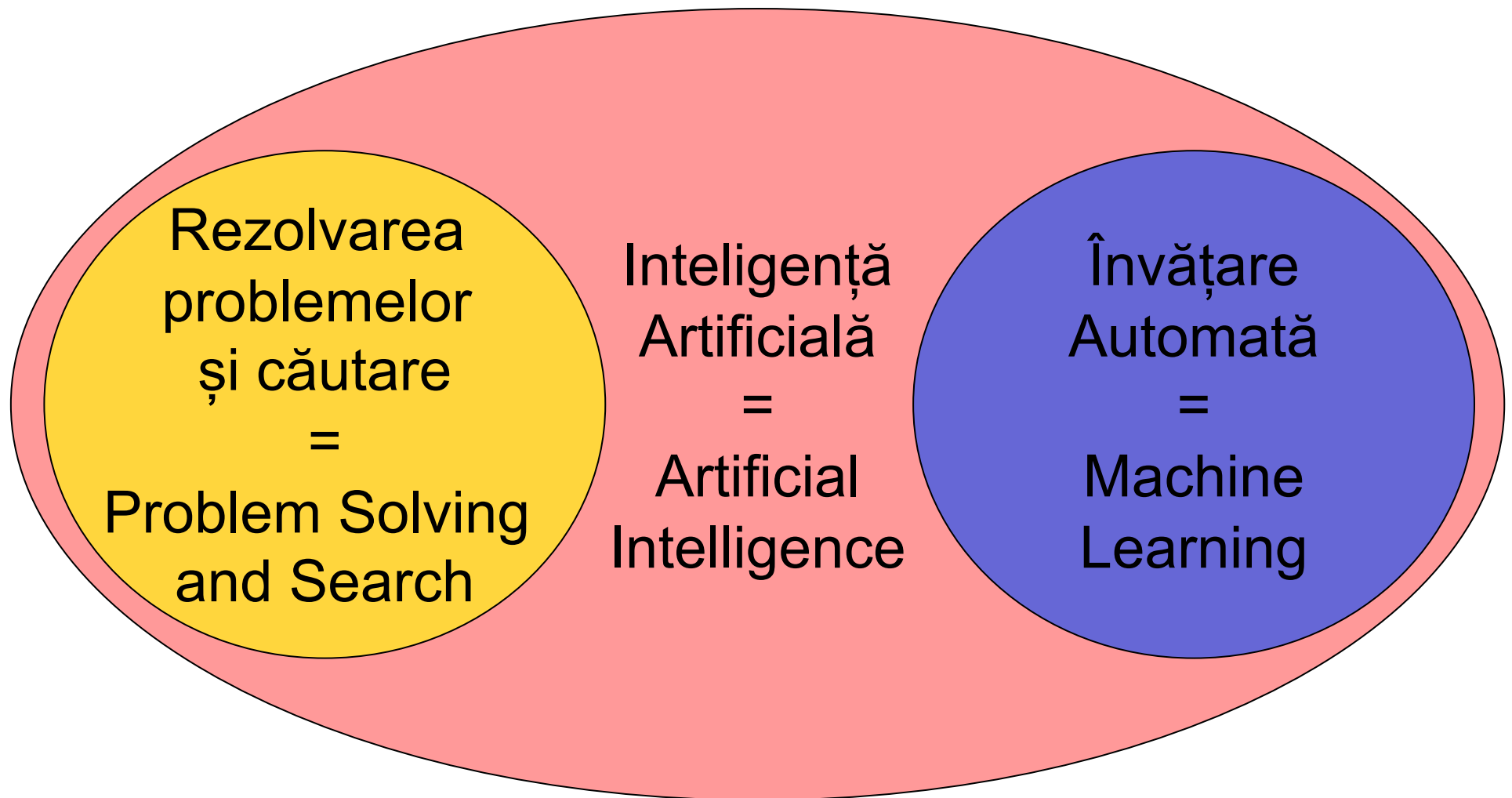
Informatică, anul II, 2025-2026

Cuprinsul cursului de azi

1. Aspecte organizatorice legate de cursul de IA
2. Prezentarea cursului de IA

Aspecte organizatorice legate de cursul de Inteligență Artificială

Structura cursului de Inteligență artificială



Echipa noastră la curs



Bogdan

Curs (50%)

Rezolvarea problemelor și căutare
(săptămânile 1-7)



Radu

Curs (50%)

Învățare automată
(săptămânile 8-14)

Echipa noastră la laborator și seminar

Rezolvarea problemelor și căutare



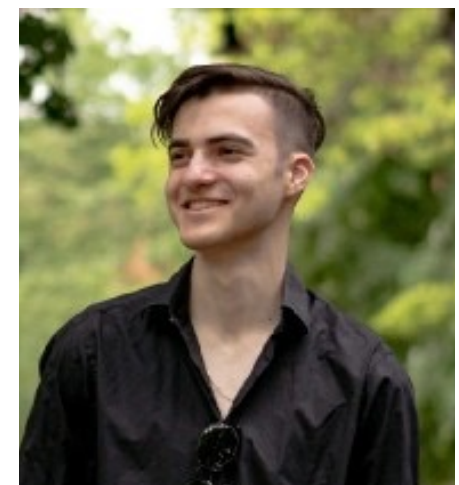
Irina

(săptămânile 1-7)
Grupele 231, 232



Miruna

(săptămânile 1-7)
Grupele 234, 251, 252



Ștefan

(săptămânile 1-7)
Grupa 233

Materiale

- pe MS-TEAMS, pe canalele Curs și Laborator+Seminar dedicate celor două părți

The screenshot shows the Microsoft Teams interface for a team named "Inteligență Artificială - 2025-2026 FMI-UB". The left sidebar contains a navigation menu with options: Home page, Class Notebook, Assignments, Insights, Classwork, Reflect, and Grades. Below this, under "Main Channels", are "General" (selected), "Învățare automată - curs", "Învățare automată - lab și sem", "Rezolvarea problemelor și căutare - curs", and "Rezolvarea problemelor și căutare - lab și s...". The main area shows the "General" channel with tabs for "General", "Posts", "Shared", and a "+" icon. A "Post in channel" box is visible with "Post" and "Announcement" options. At the bottom, a welcome message reads "Welcome to Inteligență Artificială - 2025-2026 FMI-UB" with the instruction "Choose where you want to start". Two buttons are provided: "Upload Class Materials" (with a folder icon) and "Set up Class Notebook" (with a notebook icon).

< All teams

IA

Inteligență Artificială - 2025-2026 FMI-UB

Home page

Class Notebook

Assignments

Insights

Classwork

Reflect

Grades

▼ Main Channels

General

Învățare automată - curs

Învățare automată - lab și sem

Rezolvarea problemelor și căutare - curs

Rezolvarea problemelor și căutare - lab și s...

General Posts Shared +

DA Post in channel

Post Announcement

Welcome to Inteligență Artificială - 2025-2026 FMI-UB

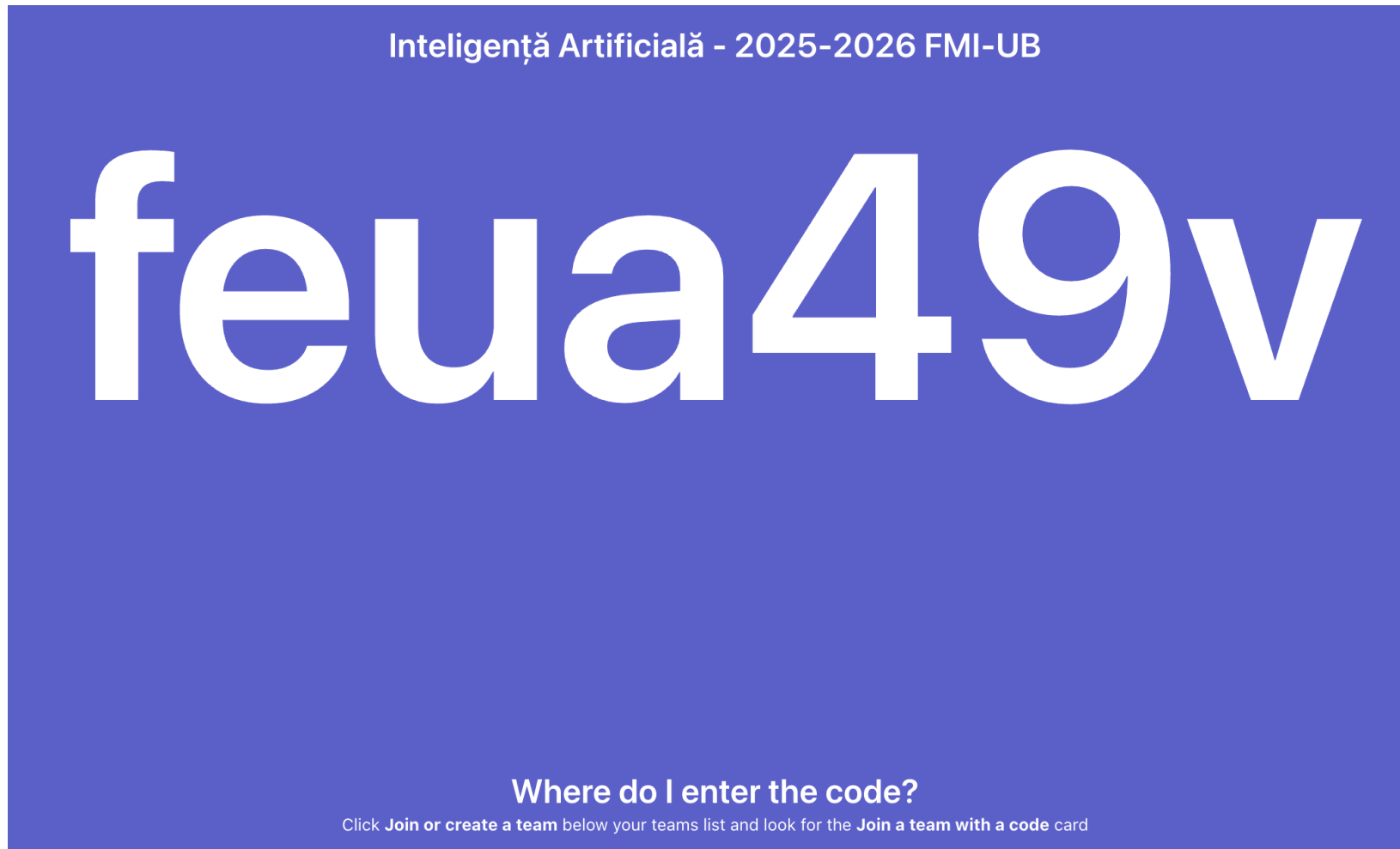
Choose where you want to start

Upload Class Materials

Set up Class Notebook

Materiale

- pe MS-TEAMS, pe canalele Curs și Laborator+Seminar dedicate celor două părți



Sistem de notare

Nota finală la disciplina Inteligență Artificială este formată din:

- nota de la curs $C = (C_1 + C_2)/2$ (evaluare scrisă) 50%
 - la examenul scris din sesiune vor fi subiecte din ambele părți
 - media notelor (C_1, C_2) din fiecare parte (între 1 – 10)
- nota de la laborator $L = (L_1 + L_2)/2$ (programare) 50%
 - vor fi două note la laborator, câte una pentru fiecare parte.
 - media notelor (L_1, L_2) din fiecare parte (între 1 – 10)
- nota finală se calculează ca media celor două note: nota de la curs și nota de la laborator = $(C + L) / 2$

Condiții de promovare: pentru a promova examenul trebuie ca $C \geq 5$ și $L \geq 5$ (regula se aplică și la restanță, **nu se reportează notele de la o sesiune la alta**)

Sistem de notare – Rezolvarea problemelor și căutare

- **bonus la Curs (C_1):**

- se acordă conform clasamentului din Kahoot (organizat la sfârșit de curs)
- seria 23 (120 de studenți) – locurile 1-4 obțin 0.3 puncte, locurile 5-8 obțin 0.2 puncte, locurile 9-12 obțin 0.1 puncte
- seria 25 (60 de studenți) – locurile 1-2 obțin 0.3 puncte, locurile 3-4 obțin 0.2 puncte, locurile 5-6 obțin 0.1 puncte
- până la 1.5 puncte în plus la nota din examen la partea de Rezolvarea problemelor și căutare (1.5 puncte din 10, fără a depăși nota 10)
- **studenții de la seria 25 nu pot primi bonus la cursul de la seria 23 și invers**

Sistem de notare – Rezolvarea problemelor și căutare

- **Laborator (L_1):**

- studenții pot opta pentru una dintre următoarele variante de evaluare:
 - A = activitate pe parcurs: $1,5p / \text{laborator} \times 6 + 1p$ oficiu.
 - P = proiect: susținut în 3 etape ($3p / \text{etapă}$) + $1p$ oficiu.
- pentru studenții care nu au obținut punctajul prin activitate/proiect se va organiza un T = test de laborator înainte de ziua examinării scrise (săptămâna 14 sau sesiune)
- bonus din Seminar (S): se acordă maximum $1p$ bonus la nota de laborator pentru activitatea din cadrul seminarelor (la prima examinare)
- $L_1 = \min(10, \max(A, P, T) + S)$

Sistem de notare – Învățare Automată

- Pentru laboratorul de "Învățare automată" nota se acordă pe baza unui test de laborator (în săptămâna a 14-a / sesiune)
- Puncte extra în timpul cursurilor / laboratoarelor
(se acordă doar la prima examinare)
- Curs:
 - Se acordă conform clasamentului din Kahoot
 - top 3 obțin 0.3 puncte pe curs, următorii 3 obțin 0.2 puncte, ș.a.m.d.
- Laborator:
 - Primul care răspunde la o întrebare / rezolvă un exercițiu primește 0.2 puncte
 - Maxim 0.4 puncte pe laborator de persoană (se punctează doar primele două răspunsuri corecte)
- Până la 1.5 puncte în plus la nota de la Curs (C_2)
- Până la 2 puncte în plus la nota din Laborator (L_2)

Sistem de notare – Învățare Automată

- Puncte extra în timpul cursurilor / laboratoarelor

(se acordă doar la prima examinare)

- Seminar:
 - Test scris la final de semestru
 - Exerciții / probleme asemănătoare cu exemplele din clasă
 - Până la 3 puncte bonus la nota de la Curs pentru testul de seminar
 - Până la 1 punct bonus pentru rezolvarea exercițiilor de seminar (0.2 per exercițiu, 0.4 maxim per student per seminar)

Sistem de notare – Învățare Automată

- Pentru nota de laborator, puteți primi până la 7 puncte bonus pentru realizarea unui proiect
- Proiectul constă în dezvoltarea unei metode de clasificare și participarea la competiția (TBA) propusă pe platforma Kaggle
- Notele vor fi proporționale cu rata de acuratețe obținută:
 - Locurile 1-10 => 7 puncte bonus
 - Locurile 11-20 => 6.5 puncte bonus
 - Locurile 21-30 => 6 puncte bonus
 - ...
 - Locurile 101-110 => 1.5 puncte bonus
 - Locurile 111-120 => 1 punct bonus
- Proiectul trebuie prezentat după ultimul laborator (se poate scădea până la 1 punct din bonificație pentru prezentare și documentație)

Sistem de notare – Învățare Automată

- Proiectele (cod+documentație) se trimit la adresa:
fmi.unibuc.ia@gmail.com
- Trimiteți doar fișiere .py! (nu se acceptă .ipynb)
- Reguli suplimentare vor fi comunicate pe pagina Kaggle a competiției
- Testul de laborator constă în implementarea și antrenarea unui anumit clasificator pe un anumit set de date, o parte semnificativă din nota obținută fiind proporțională cu acuratețea obținută de clasificatorul antrenat.
- Pentru atingerea punctului optim de învățare, punctajul va fi maxim. Vor exista câteva cerințe (cu punctaj separat) care să vă ajute la găsirea modelului și hiperparametrilor optimi.

Sistem de notare – Învățare Automată

- Codul de la proiecte va fi verificat cu soft-uri anti-plagiat
- NU este permisă preluare codului de pe web (sub nicio formă, nici măcar din ...)
- NU este permisă preluare codului de la colegi
- Bonificația poate fi retrasă pe baza similarităților identificate

Exemple de plagiarism

```
3
# average test loss
test_loss = test_loss/len(validloader.dataset)
print('Test Loss: {:.6f}\n'.format(test_loss))

for i in range(3):
    if class_total[i] > 0:
        print('Test Accuracy of %5s: %2d%% (%2d/%2d)' % (
            classes[i], 100 * class_correct[i] / class_total[i],
            np.sum(class_correct[i]), np.sum(class_total[i])))
    else:
        print('Test Accuracy of %5s: N/A (no training examples)' % (classes[i]))

print('\nTest Accuracy (Overall): %2d%% (%2d/%2d)' % (
    100. * np.sum(class_correct) / np.sum(class_total),
    np.sum(class_correct), np.sum(class_total)))
```

Exemple de plagiarism

4

```
print('Epoch: {} \tTraining Loss: {:.6f} \tValidation Loss: {:.6f}'.format(
    epoch, train_loss, valid_loss))
```

```
# save model if validation loss has decreased
```

```
if valid_loss <= valid_loss_min:
```

```
    print('Validation loss decreased ({:.6f} --> {:.6f}). Saving model ...'.format(
        valid_loss_min,
        valid_loss))
```

```
    torch.save(model.state_dict(), 'model_curent.pt')
```

```
    valid_loss_min = valid_loss
```

```
model.load_state_dict(torch.load('model_curent.pt'))
```

Exemple de plagiarism

```
batch_size = 64
```

```
1 for data, target in validloader:
```

```
    # move tensors to GPU if CUDA is available
```

```
    if train_on_gpu:
```

```
        data, target = data.cuda(), target.cuda()
```

```
    # forward pass: compute predicted outputs by passing inputs to the model
```

```
    output = model(data)
```

```
    # calculate the batch loss
```

```
    loss = criterion(output, target)
```

```
    # update test loss
```

```
    test_loss += loss.item()*data.size(0)
```

```
    # convert output probabilities to predicted class
```

```
    _, pred = torch.max(output, 1)
```

```
    # compare predictions to true label
```

```
    correct_tensor = pred.eq(target.data.view_as(pred))
```

```
    correct = np.squeeze(correct_tensor.numpy()) if not train_on_gpu else  
np.squeeze(correct_tensor.cpu().numpy())
```

Exemple de plagiarism

```
#####
```

```
# validate the model #
```

```
#####
```

```
1 model.eval()
```

```
for data, target in validloader:
```

```
    # move tensors to GPU if CUDA is available
```

```
    if train_on_gpu:
```

```
        data, target = data.cuda(), target.cuda()
```

```
    # forward pass: compute predicted outputs by passing inputs to the model
```

```
    output = model(data)
```

```
    # calculate the batch loss
```

```
    loss = criterion(output, target)
```

```
    # update average validation loss
```

```
    valid_loss += loss.item() * data.size(0)
```


Exemple de plagiarism

```
def forward(self, x):  
    x = F.relu(F.max_pool2d(self.conv1(x), 2))  
    x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))  
    x = F.relu(F.max_pool2d(self.conv2_drop(self.conv3(x)), 2))  
    x = x.view(x.shape[0], -1)  
    x = F.relu(self.fc1(x))  
    x = F.dropout(x, training=self.training)  
    x = self.fc2(x)  
    x = F.dropout(x, training=self.training)  
    x = self.fc3(x)  
    return x
```

Exemple de plagiarism

```
1 model.eval()
for data, target in validloader:
    # move tensors to GPU if CUDA is available
    if train_on_gpu:
        data, target = data.cuda(), target.cuda()
    # forward pass: compute predicted outputs by passing inputs to the model
    output = model(data)
    # calculate the batch loss
    loss = criterion(output, target)
    # update average validation loss
    valid_loss += loss.item() * data.size(0)

    1 _, pred = torch.max(output, 1)
    # compare predictions to true label
    correct_tensor = pred.eq(target.data.view_as(pred))
    correct = np.squeeze(correct_tensor.numpy()) if not train_on_gpu else
np.squeeze(correct_tensor.cpu().numpy())
```

Exemple acceptable

```
3 from keras.layers import Conv2D, Activation, MaxPooling2D, Flatten, Dense, Dropout
from keras.models import Sequential
from pandas import read_csv
from sklearn.metrics import confusion_matrix
from tqdm import tqdm
from keras.preprocessing import image
11 from keras.utils.np_utils import to_categorical
import numpy as np
import plot as plt
```

Exemple acceptable

```
    imagini_validare.append(imagine)
    imagini_train = np.array(imagini_train)
    imagini_validare = np.array(imagini_validare)
    2 train_labels = np.array(train_labels)
    validation_labels = np.array(validation_labels)

nume_imagini = []
i = 0
ordine = dict()
    9 for numeImagine in os.listdir(PATH + "/test"):
        imagine = Image.open(PATH + "/test/" + numeImagine)
        #imagine = imagine.convert('RGB')
        imagine = np.array(imagine).astype('d')
        imagini_test.append(imagine)
        ordine[numeImagine] = i
        i += 1
        nume_imagini.append(numeImagine)
    imagini_test = np.array(imagini_test)
    11 imagini_train = np.repeat(imagini_train[..., np.newaxis], 3, -1)
    imagini_test = np.repeat(imagini_test[..., np.newaxis], 3, -1)
    imagini_validare = np.repeat(imagini_validare[..., np.newaxis], 3, -1)
```

Rotunjirea notelor obținute

- orice notă $x < 5$ se rotunjește la partea sa întreagă $[x]$
 - nota 4.99 se rotunjește la nota 4;
 - nota 3.4 se rotunjește la nota 3.
- pentru orice notă $x \geq 5$:
 - dacă partea fracționară $\{x\} \geq 0.5$ atunci rotunjim la $[x] + 1$
 - nota 9.5 se rotunjește la nota 10;
 - dacă partea fracționară $\{x\} < 0.5$ atunci rotunjim la $[x]$
 - nota 9.45 se rotunjește la nota 9;

Regulament de integritate

- regulament privind activitatea studenților la UB:

http://fmi.unibuc.ro/ro/pdf/2015/consiliu/UB_Regulament_studenti_2015.pdf

- regulament de etică și profesionalism la

FMI:http://fmi.unibuc.ro/ro/pdf/2015/consiliu/Regulament_etica_FMI.pdf

Se consideră **incident minor** cazul în care un student/ o studentă:

- a. preia codul sursă/ rezolvarea unei teme de la un coleg/ o colegă și pretinde că este rezultatul efortului propriu;

Se consideră **incident major** cazul în care un student/ o studentă:

- a. copiază la examene de orice tip;

Prezentarea cursului de Inteligență Artificială

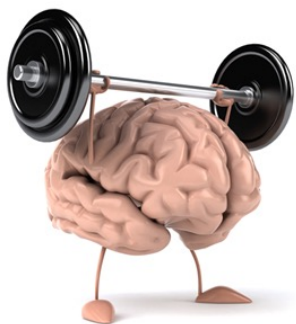
Homo sapiens

- omul inteligent;
- putem percepe, înțelege lumea care ne înconjoară, putem manipula obiecte și realiza predicții despre ce se va întâmpla în mediul în care suntem;
- ne dezvoltăm inteligența de-a lungul vieții pe baza învățării din experiențe, ne ajută la rezolvarea de probleme;
- putem realiza fără efort sarcini care pentru un computer sunt la acest moment încă dificil de realizat: descrierea în limbaj natural a ceea ce se întâmplă în jurul nostru, interacțiune cu mediul înconjurător (obiecte, persoane), comunicarea cu ceilalți, etc.

Agenți inteligenți

- agent = entitate care percepe și acționează într-un anumit mediu
- exemple: om, robot de curățenie, mașină autonomă, program software care detectează spam-urile în Inbox, program software care detectează fețele oamenilor într-o imagine
- agent inteligent: agenți înzestrați cu o “intelență” care îi ajută să ia deciziile optime în mediile lor, asemenea oamenilor

Inteligență Artificială – definiție



- Inteligență - abilitatea de a rezolva probleme

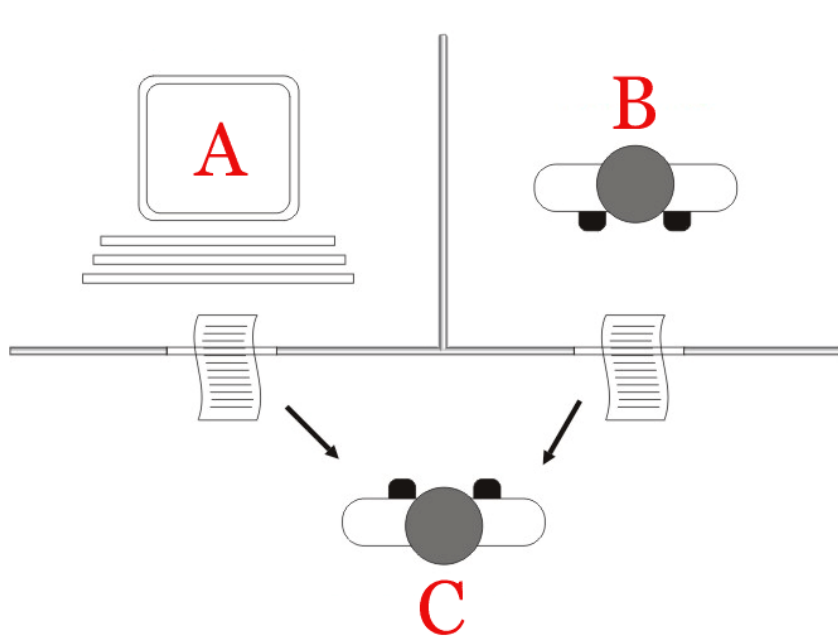


- Inteligență artificială - construirea de mașini / scrierea de programe software inteligente pentru rezolvarea de probleme sau luarea deciziilor în mod automat în situații în care oamenii și-ar folosi inteligența

INTELIGENȚĂ, *inteligente*, s. f. 1. Capacitatea de a înțelege ușor și bine, de a sesiza ceea ce este esențial, de a rezolva situații sau probleme noi pe baza experienței acumulate anterior; deșteptăciune. ♦ Inteligență artificială = domeniu al informaticii care dezvoltă sisteme tehnice capabile să rezolve probleme dificile legate de inteligența umană. ♦ Persoană inteligentă. 2. (în forma *intelligenție*) Totalitatea intelectualilor; intelectualitate (2). [Var.: inteligință, intelighenție s. f.] – Din fr. *intelligence*, lat. *intelligentia*, germ. *Intelligenz*, rus. *inteligencia*.

Scopul inteligenței artificiale

- Scopul suprem al inteligenței artificiale este de a construi sisteme (agenți) care să atingă/depășească nivelul de inteligență al omului
- Testul Turing: un computer prezintă un nivel de inteligență uman dacă un interlocutor uman nu reușește să distingă, în urma unei conversații în limbaj natural, că vorbește cu un om sau cu un calculator



Două paradigme fundamentale în Inteligența Artificială

Inteligența Artificială a evoluat istoric în jurul a două idei majore:

1. Paradigma **simbolică** în care inteligența = raționament explicit (căutare, planificare, jocuri)

- ideea centrală: dacă știm regulile problemei, putem căuta soluția într-un spațiu al stărilor → Rezolvarea problemelor și căutare

2. Paradigma **statistică**, bazată pe date în care inteligența = învățare din experiență

- ideea centrală: dacă nu știm regulile problemei, putem căuta soluția învățând din date → Învățare automată

Ambele rezolvă același tip de problemă generală: cum construim agenți inteligenți.

Două paradigme fundamentale în Inteligența Artificială

La acest curs vom studia două moduri fundamentale prin care un sistem poate deveni inteligent:

- prin căutare într-un spațiu (de stări) bine definit.
- prin învățare din date atunci când regulile nu sunt cunoscute explicit.

În acest curs vom înțelege ambele mecanisme și vom vedea că, în esență, ele rezolvă aceeași problemă: optimizarea comportamentului unui agent într-un mediu.

Scurt istoric al inteligenței artificiale



A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.

(John McCarthy)



Scurt istoric al inteligenței artificiale

- “We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire.”
- The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.
- An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves.
- We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.”

Scurt istoric al inteligenței artificiale

- Anii 1960-1980: "AI Winter"
- Anii 1990: Rețelele neuronale domină, în principal datorită descoperirii algoritmului de propagare a erorii înapoi pentru rețele cu mai multe straturi
- Anii 2000: Metodele kernel domină, în principal din cauza instabilității rețelelor neuronale
- Anii 2010: Revenirea la rețele neuronale, în principal datorită conceptului de învățare profundă (deep learning)

Inteligența Artificială la INFO/FMI

- cursuri legate de Inteligența Artificială în anul 3 și la master

Semestrul 1:

Specialitate

Baze de date, de la NoSql la Vector DBs

[industrie] Data Oriented Design în Dezvoltarea de Jocuri

Inteligența Artificială în Educație

Introducere în Cluster Computing - sem1

Implementarea concurenței în limbaje de programare

Arheologia mașinilor inteligente

Tehnologii Cloud Computing Aplicate în Machine Learning

Tehnici de simulare

Blockchain - concepte, tehnologii și aplicații

Managementul amenințărilor cibernetice

Prelucrarea Semnalelor

Sisteme distribuite

Calcul Numeric în Informatică

[industrie] Dezvoltarea jocurilor 3D folosind Unreal Engine 5

Introducere în Reinforcement Learning

Grafica pe calculator

Concepte și aplicații în Vederea Artificială

Introducere în robotica

[industrie] Robotic Process Automation (RPA) folosind platforma UiPath

[industrie] Java Script server side: NodeJS & GraphQL

Introducere în programarea jocurilor pe calculator

Inteligența Artificială la INFO/FMI

- cursuri legate de Inteligența Artificială în anul 3 și la master

Semestrul 2:

Specialitate

Securizarea și Automatizarea Rețelelor pentru Întreprinderi

Introducere în evaluarea performanțelor sistemelor informatice

Introducere în Cluster Computing - sem2

Tehnologii new media

Inteligența artificială neuro-simbolică

Metode Formale în Ingineria Software

Dezvoltarea aplicațiilor software securizate

Tehnici de Optimizare

Algoritmica avansată a grafurilor

Introducere în cercetare și bioinformatică

Strategii de planificare a unei echipe de roboți (SPER)

[industrie] Învățare automată în Vedere Artificială

Testarea Sistemelor Software

Programarea aplicațiilor de simulare

[industrie] Învățarea rețelelor neurale adânci (Intensiv)

Protocoale criptografice

[industrie] Production engineering

Fundamentele proiectării compilatoarelor

Sistemul de operare Linux

Elemente de securitate și logică aplicată

Introducere în prelucrarea limbajului natural

Ce veți studia la acest curs în prima parte?

1. Rezolvarea problemelor prin căutare

Stare

Scop

Funcție succesor

Acțiuni

Spațiul stărilor

Arbore de căutare

Căutare neinformată

Căutare informată

2. Strategii de căutare neinformate

Căutare în lățime

Căutare în adâncime

Căutare cu cost uniform

Căutare cu adâncime limitată

Căutare cu adâncime incrementală

3. Strategii de căutare informate

Euristică

Greedy

A*

Euristică admisibilă

4. Strategii de căutare adversariale

Algoritmul minimax

Alpha-Beta retezare

Algoritmul expectimax

5. Algoritmi genetici

individ

populație

funcție de fitness

încrucișare

mutație

Problema 8-puzzle

Descrierea problemei

Pe o tablă 3×3 se găsesc 8 piese, numerotate de la 1 la 8. La fiecare moment o singură piesă se poate mișca cu o poziție, pe orizontală sau verticală, în limitele cadrului tablei, în locul rămas liber (poziția roșie). Se dă o configurație inițială și una finală a tablei. Trebuie să se găsească secvența de mutări care să aducă piesele din configurația inițială în cea finală.

2		3
1	8	4
7	6	5

Stare inițială

[2, X, 3, 1, 8, 4, 7, 6, 5]



1	2	3
8		4
7	6	5

Stare scop

[1, 2, 3, 8, X, 4, 7, 6, 5]

Formularea problemei 8-puzzle

- Spațiul stărilor

- stare = configurație a pătratului 3×3
- $9!/2$ stări posibile
- stare inițială
- stare scop

2		3
1	8	4
7	6	5

Stare inițială

[2, X, 3, 1, 8, 4, 7, 6, 5]

1	2	3
8		4
7	6	5

Stare scop

[1, 2, 3, 8, X, 4, 7, 6, 5]

- Acțiuni

- STÂNGA, JOS, DREAPTA, SUS
- fiecare acțiune (mutare) are cost = 1

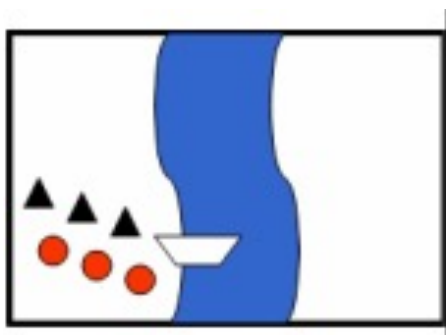
- Funcția succesori

- $SUCCESSOR([X, a, b, c, d, e, f, g, h], STANGA) = \text{NU EXISTA}$
- $SUCCESSOR([X, a, b, c, d, e, f, g, h], SUS) = \text{NU EXISTA}$
- $SUCCESSOR([X, a, b, c, d, e, f, g, h], DREAPTA) = [a, X, b, c, d, e, f, g, h]$
- $SUCCESSOR([X, a, b, c, d, e, f, g, h], JOS) = [c, a, b, X, d, e, f, g, h]$
- ...

Problema misionarilor și a canibalilor

Descrierea problemei

Trei misionari și **trei canibali** se află la marginea unui râu, cu scopul de a trece pe celălalt mal. Ei au la dispoziție o barcă de două persoane. Dacă la un moment dat, pe un mal sau pe celălalt, numărul canibalilor este mai mare decât cel al misionarilor, canibalii îi vor mânca pe misionari. Problema constă în a afla cum pot trece cele 6 persoane în deplină siguranță de pe un mal pe celălalt☺. Barca nu merge singură, este nevoie de cel puțin o persoană în barcă.



Stare inițială

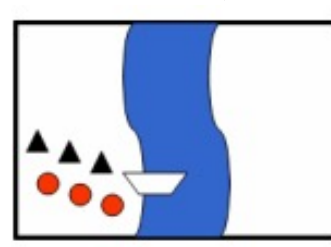


Stare scop

Formularea problemei misionarilor și a canibalilor

- Spațiul stărilor

- triplet (m, c, b) : m = numărul de misionari pe malul stâng, c = numărul de canibali pe malul stâng, b = prezența bărcii (0 sau 1) pe malul stâng
- stare inițială: $(3,3,1)$
- stare scop: $(0,0,0)$



Stare inițială



Stare scop

- Acțiuni

- pot duce de pe un mal către celălalt mal 1 sau 2 misionari, 1 sau 2 canibali, 1 misionar + 1 canibal ($1m, 2m, 1c, 2c, 1m+1c$)

- Funcția succesor

- $\text{Sucesor}((m, c, b), 1c) = (m, c + 1 - 2b, 1 - b), \dots$
- $\text{Sucesor}((m, c, b), 1m) = (m + 1 - 2b, c, 1 - b), \dots$

Arbore de joc pentru X și 0



MAX (X)



MIN (O)



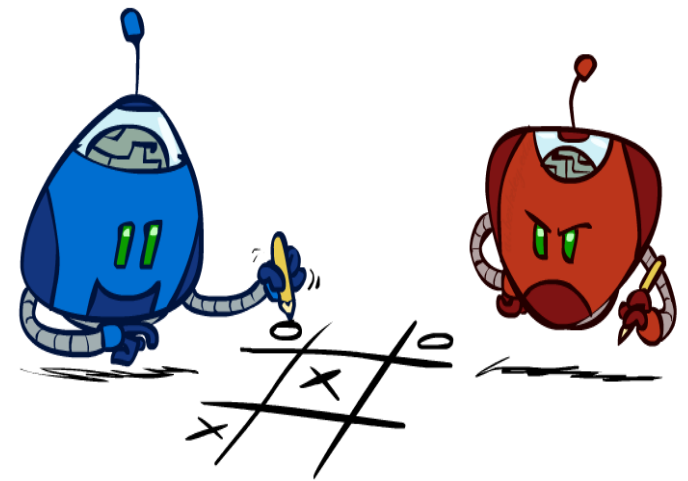
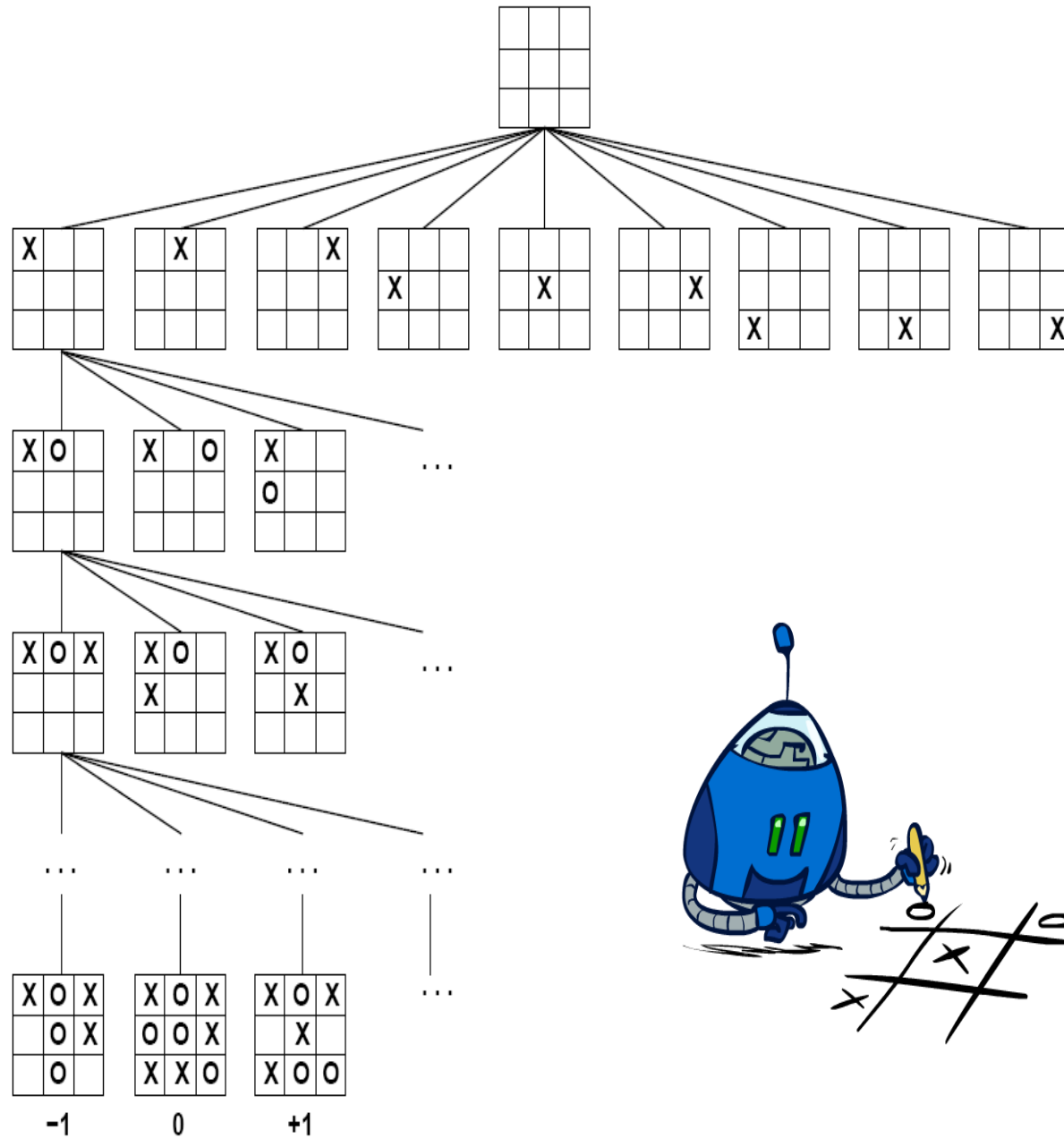
MAX (X)



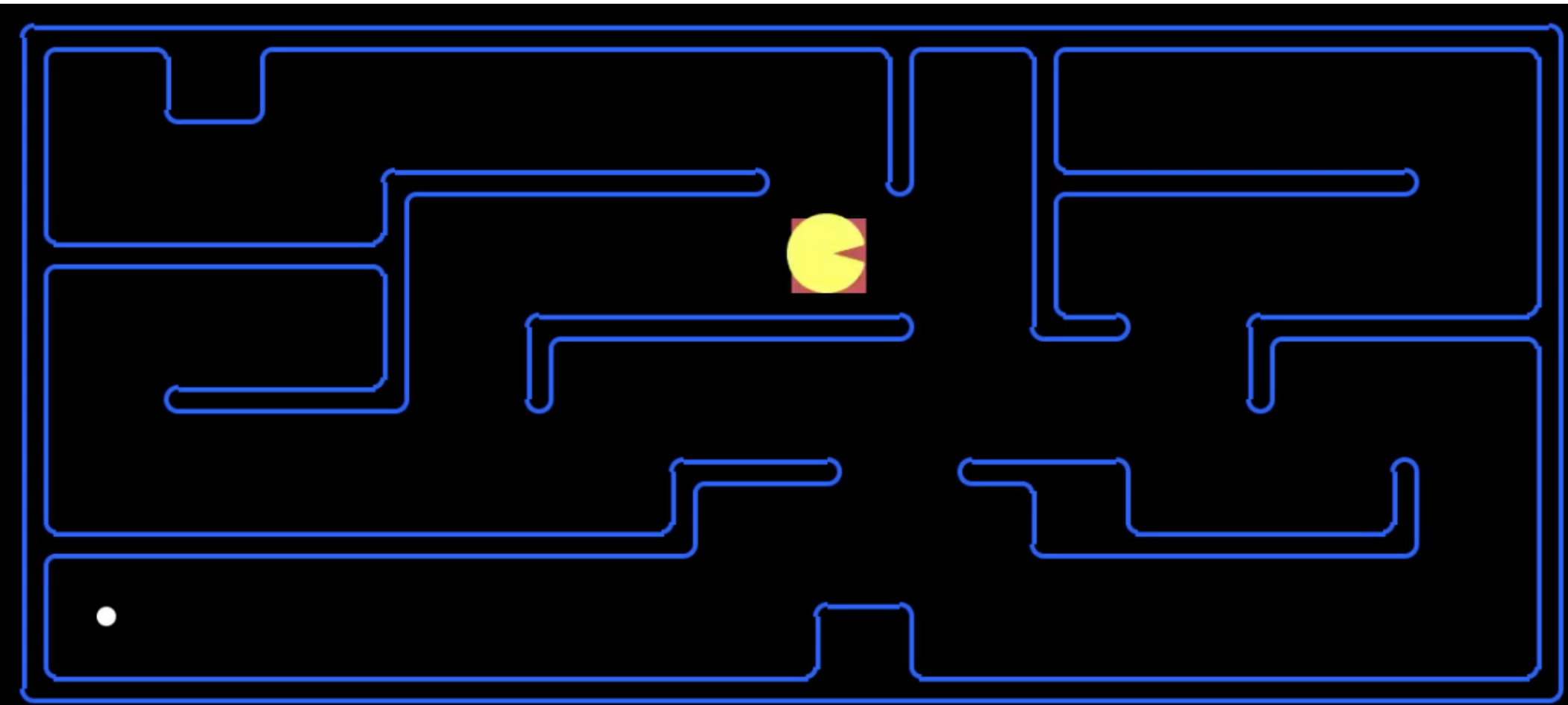
MIN (O)

TERMINAL

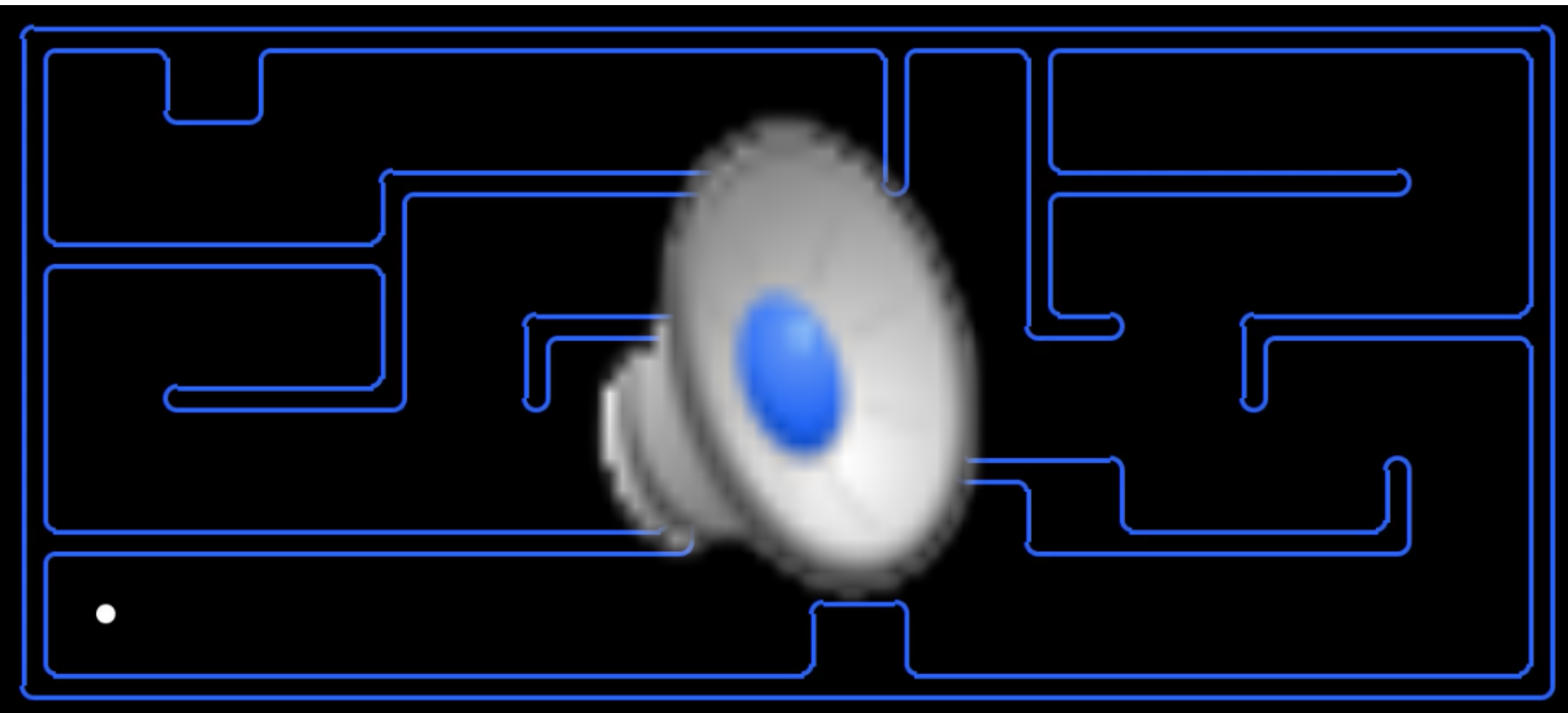
Utility



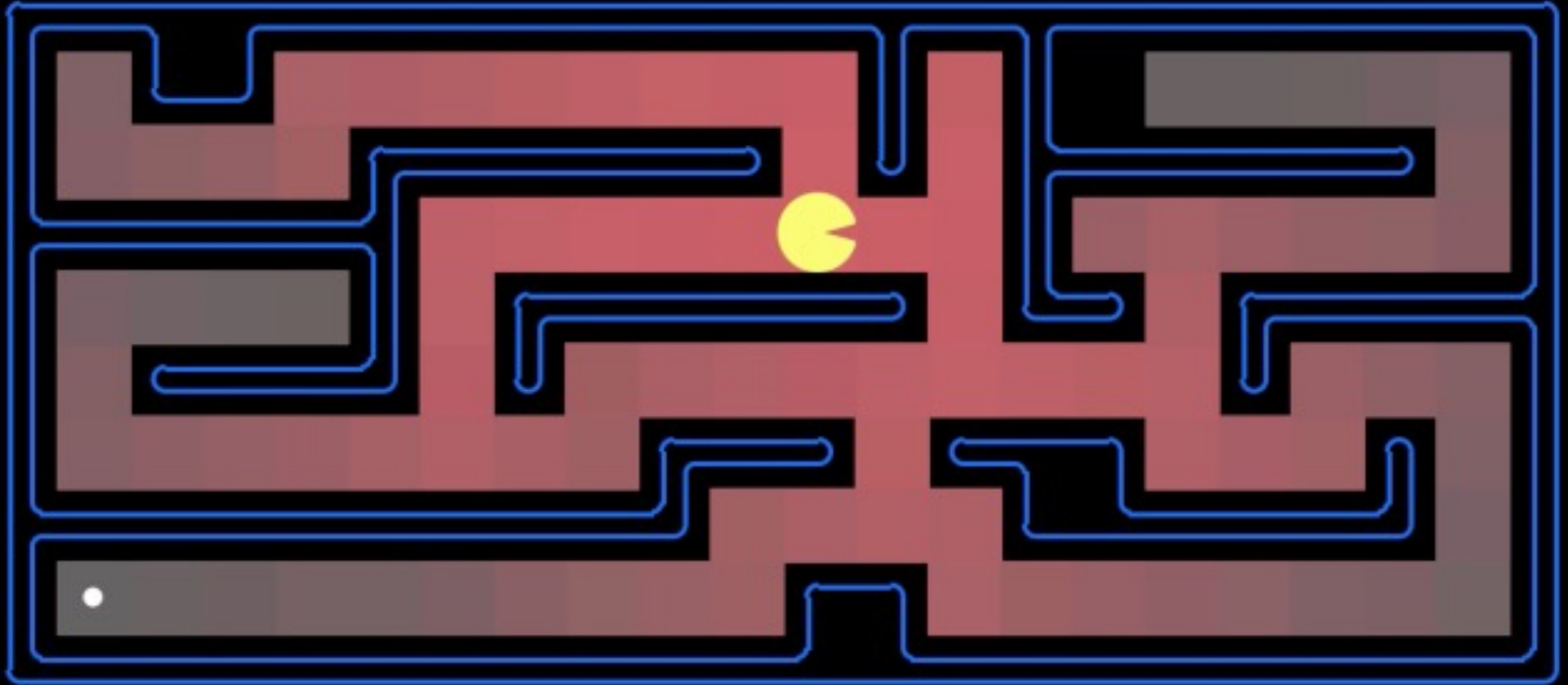
Explorarea arborelui de căutare



Explorarea arborelui de căutare



Explorarea arborelui de căutare



SCORE: 0

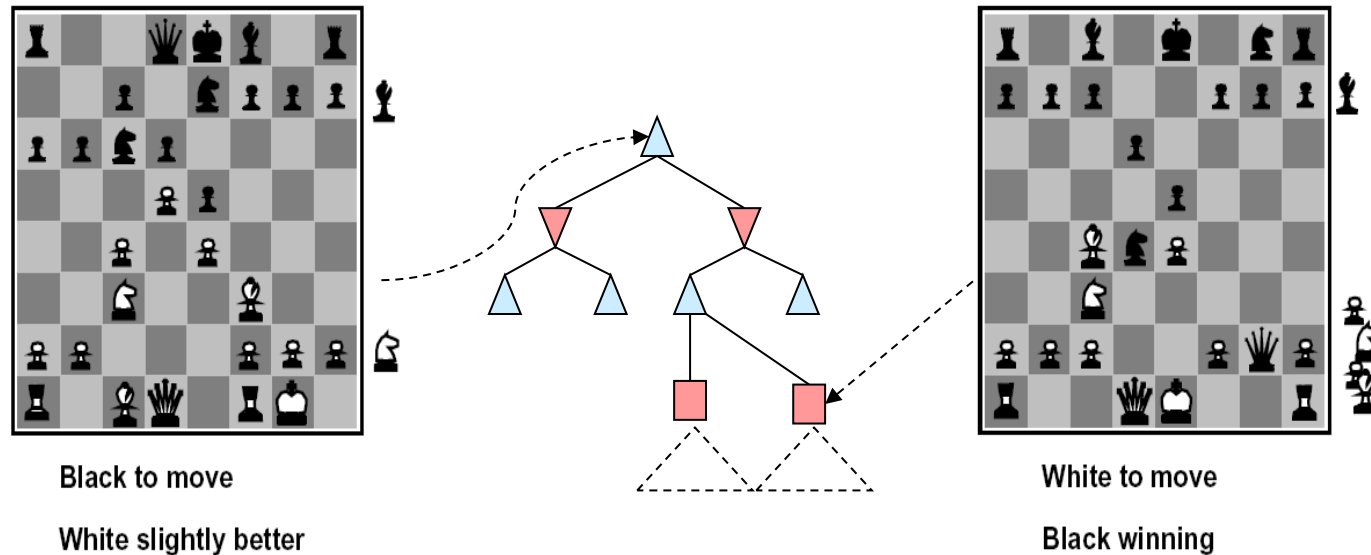
Ordinea de explorare a nodurile este dată de culoare:

Roșu intens – noduri explorate la început

Gri – noduri explorate la sfârșit

Funcții de evaluare - șah

- Funcțiile de evaluare asociază un scor stărilor neterminale



- o funcție de evaluare ideală returnează valoarea MiniMax a stării.
- diverse funcții de evaluare specifice pentru fiecare joc: pentru șah se consideră funcții liniare în care ponderăm piesele de pe tablă

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- unde, $f_1(s) = (\text{\#regine_albe} - \text{\#regine_negre})$, $w_1 = 100$, etc.

Ce veți studia la acest curs în a doua parte?

- Scopul suprem al inteligenței artificiale este de a construi sisteme (agenți) care să atingă nivelul de inteligență al omului
- O mare parte din cercetători consideră că acest scop poate fi atins prin imitarea modului în care oamenii învață
- **Învățarea automată** – domeniu care studiază modul în care calculatoarele / mașinile pot fi înzestrate cu abilitatea de a învăța, fără ca aceasta să fie programată în mod explicit
- În acest context, **învățarea** se referă la:
 - recunoașterea unor tipare / structuri (patterns) complexe
 - luarea deciziilor inteligente bazate pe observațiile din **date**

Problemă “bine pusă” de învățare automată

- Ce probleme pot fi rezolvate* folosind învățarea automată?
- **Problemă “bine pusă” de învățare automată:**
- Spunem despre un program pe calculator că învață dintr-o experiență E în raport cu o clasă de task-uri T și o măsură de performanță P , dacă performanța sa în rezolvarea task-urilor T , măsurată prin P , se îmbunătățește odată cu experiența E
- (*) rezolvate cu un anumit grad de acuratețe

Problemă “bine pusă” de învățare automată

- Arthur Samuel (1959) a scris un program pentru a juca dame (probabil primul program bazat pe conceptul de învățare)
- Programul a jucat împotriva lui însuși 10 mii de jocuri
- Programul a fost conceput să găsească ce poziții ale tablei de joc erau bune sau rele în funcție de probabilitatea de a câștiga sau pierde
- În acest caz:
 - $E = 10000$ de jocuri
 - $T =$ joacă dame
 - $P =$ dacă câștigă sau nu



Strong AI versus Weak AI

-  • ***strong / generic / true AI (vezi definiția lui Turing)***
 - mașini care gândesc la un nivel apropiat sau superior oamenilor
 - nu avem așa ceva în zilele noastre./?
- **weak / narrow AI (se focusează pe o anumită problemă)**
 - simulează inteligența artificială
 -  • se bazează pe detectare de pattern-uri
 - paradigma dominantă azi în inteligența artificială

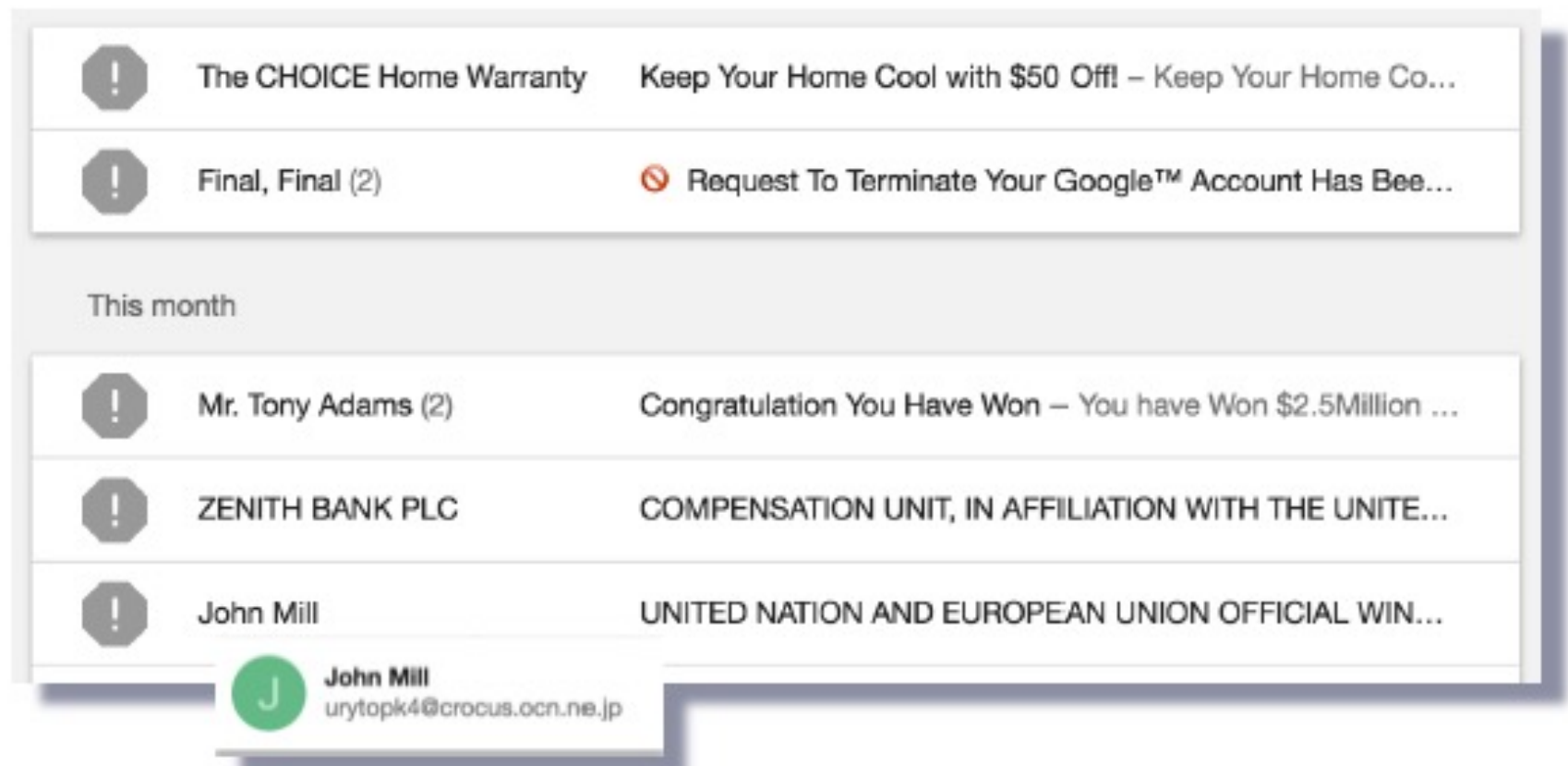
Învățarea automată

- Se aplică în situații în care este foarte greu (imposibil) să definim un set de reguli de mână / să scriem un program
- Cum ați scrie un program pe calculator care vrea să găsească mașinile într-o imagine?



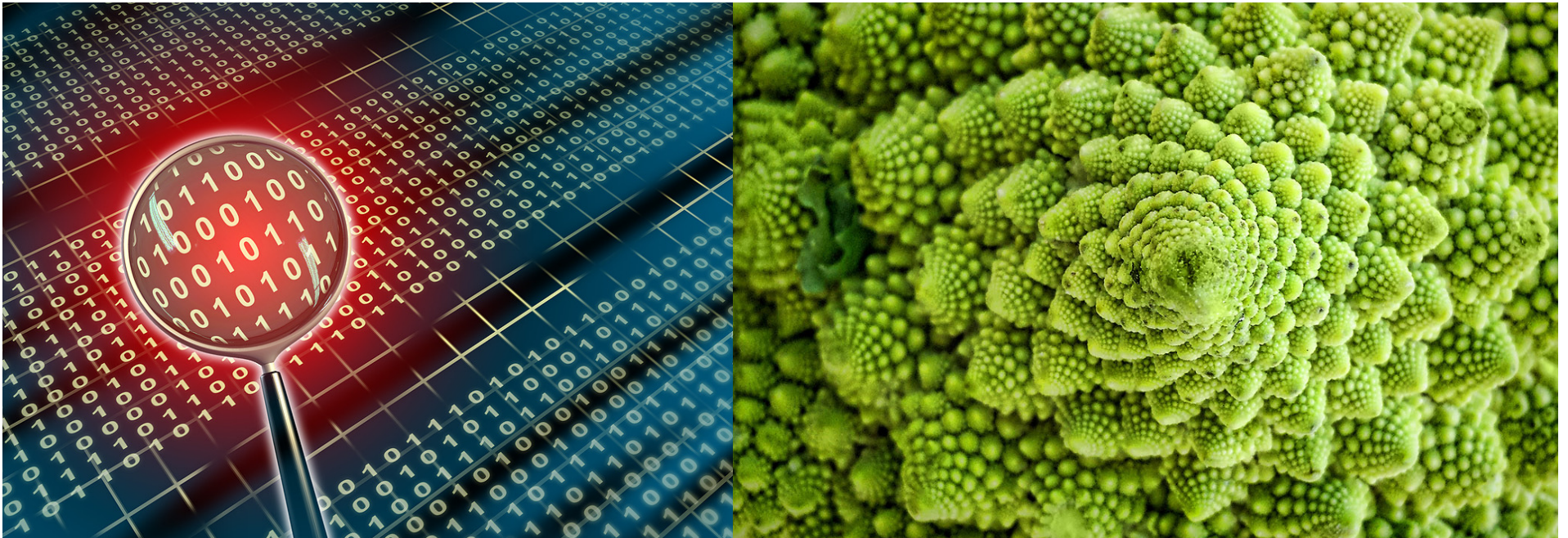
Învățarea automată

- Se aplică în situații în care este foarte greu (imposibil) să definim un set de reguli de mână / să scriem un program
- Cum ați scrie un program pe calculator care să filtreze mailurile spam?



Esența învățării automate

- Există un tipar
- Dar nu îl putem exprima programatic / matematic
- Avem date / exemple în care regăsim acest tipar



Traditional Programming



Machine Learning



Ce este învățarea automată?

[Arthur Samuel, 1959] field of study that

- gives computers the ability to learn without being explicitly programmed

[Kevin Murphy] algorithms that

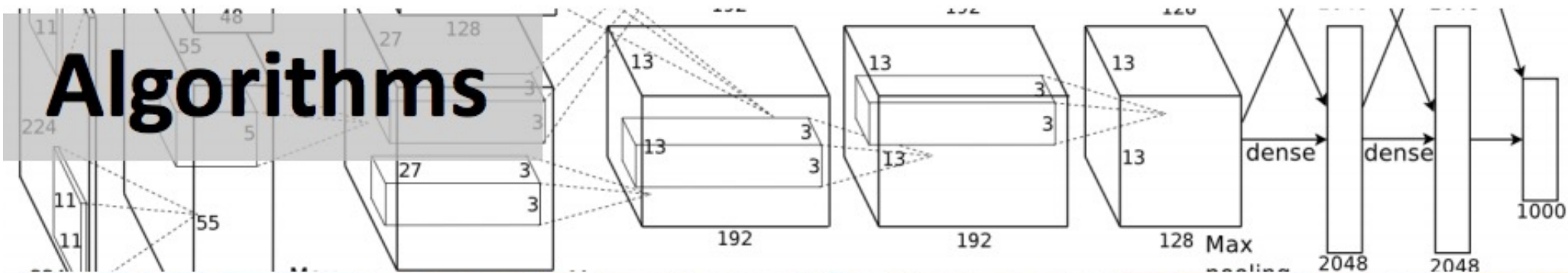
- automatically detect patterns in data
- use the uncovered patterns to predict future data or other outcomes of interest

[Tom Mitchell] algorithms that

- improve their performance (P)
- at some task (T)
- with experience (E)

Ingrediente pentru Învățare Automată

Algorithms



Data

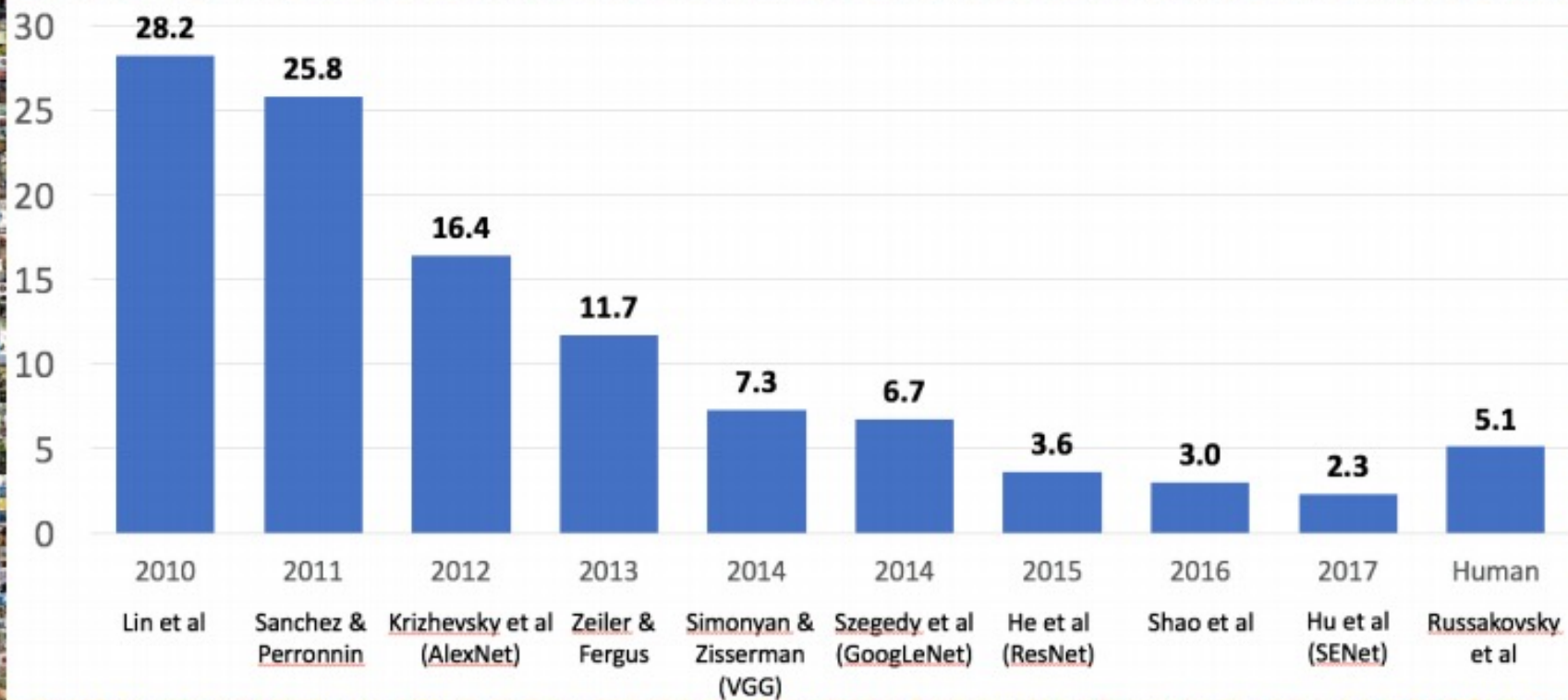


Computation



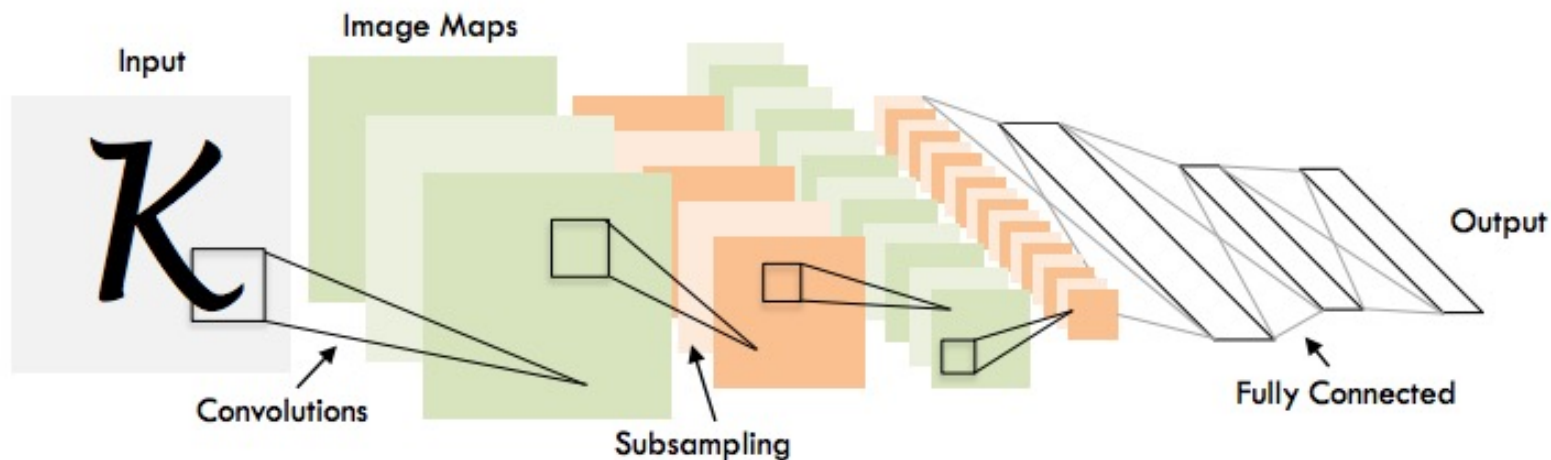
IMAGENET Large Scale Visual Recognition Challenge

**Clasificarea imaginilor
1,431,167 imagini adnotate cu
1000 clase de obiecte**



1998

LeCun et al.



of transistors



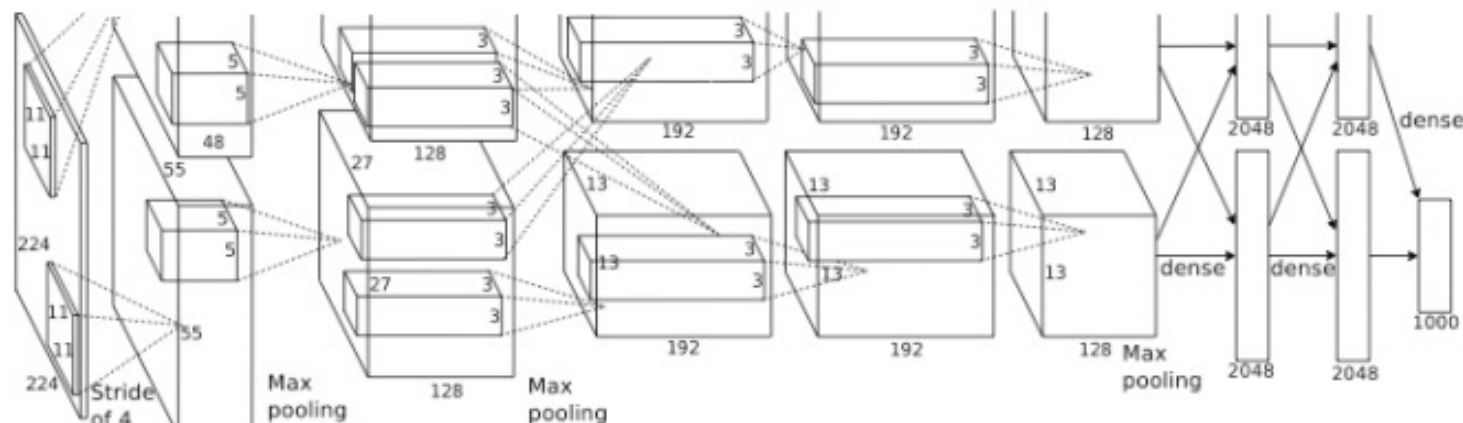
10^6

of pixels used in training

10^7 **NIST**

2012

Krizhevsky et al.



of transistors



10^9

GPUs



of pixels used in training

10^{14} **IMAGENET**

Esența învățării automate

- Mii de algoritmi de învățare automata existenți
 - Cercetătorii publică sute de noi algoritmi în fiecare an
- Simplificând decenii de cercetare în domeniu, putem reduce învățarea automată la:
 - Învățarea unei funcții f care să mapeze un input X către un output Y , anume $f: X \rightarrow Y$
 - Exemplu: X : email-uri, Y : {spam, non-spam}

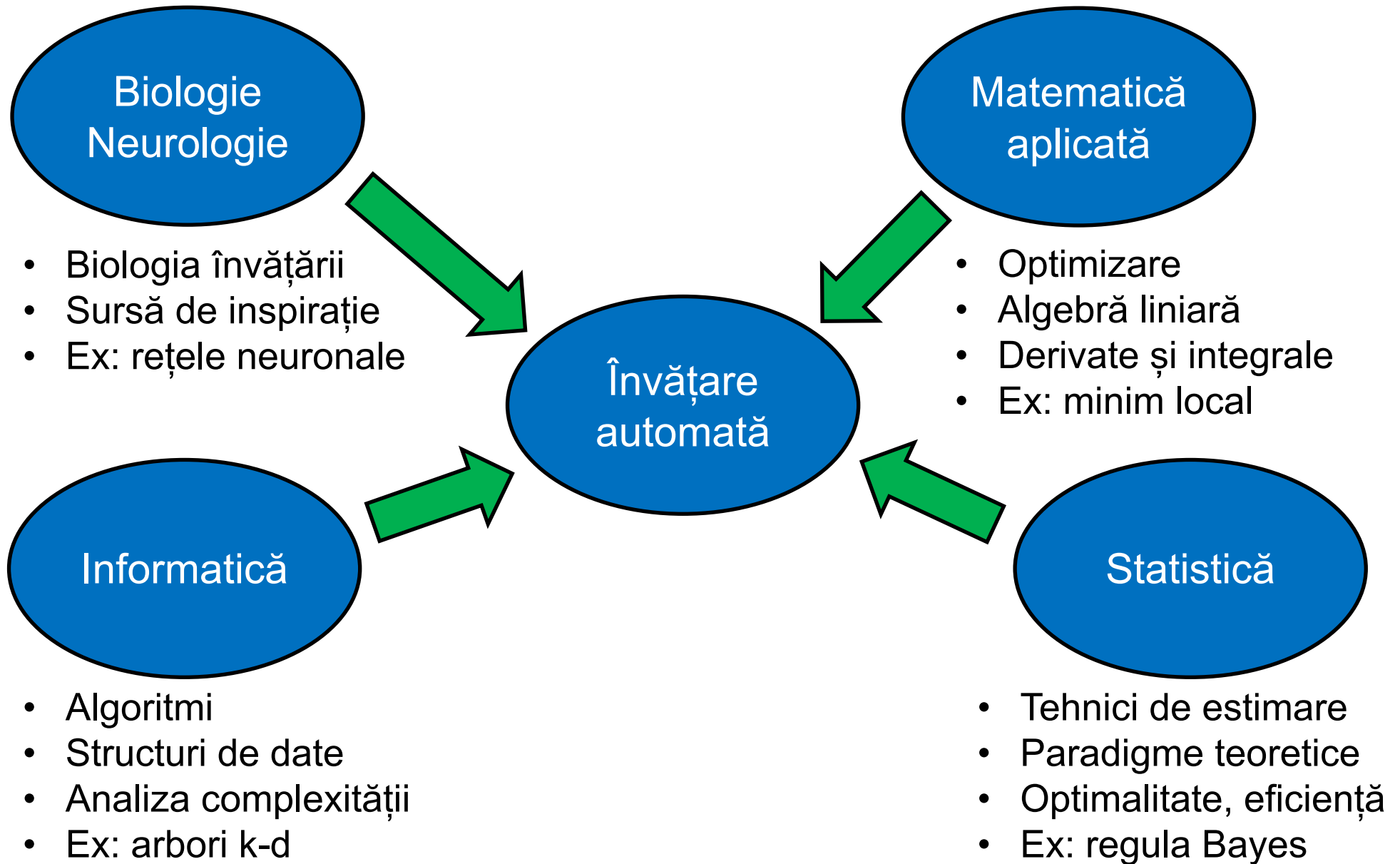
Esența învățării automate

- Input: X (imagini, texte, email-uri...)
- Output: Y (spam sau non-spam...)
- Funcție Target (necunoscută)
 $f: X \rightarrow Y$ (realitatea / "adevărata" mapare)
- Date
 $(x_1, y_1), (x_2, y_2), \dots (x_N, y_N)$
- Model
 $g: X \rightarrow Y$
 $y = g(x) = \text{sign}(w^T x)$

Esența învățării automate

- Orice algoritm de învățare automată are 3 componente:
 - Reprezentare / Modelare
 - Evaluare / Funcție obiectiv
 - Optimizare

Ce cunoștințe sunt necesare?



Paradigme ale învățării

- Învățare supervizată (supervised learning)
- Învățare nesupervizată (unsupervised learning)
- Învățare semi-supervizată (semi-supervised learning)
- Învățare ranforsată (reinforcement learning)

Paradigme non-standard:

- Învățarea activă (active learning)
- Învățare prin transfer (transfer learning)

Învățare supervizată

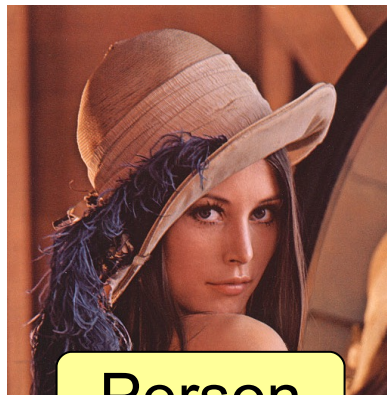
- Avem la dispoziție exemple de obiecte etichetate
- Exemplu 1: recunoașterea obiectelor din imagini cu eticheta obiectelor conținute



Car



Car



Person



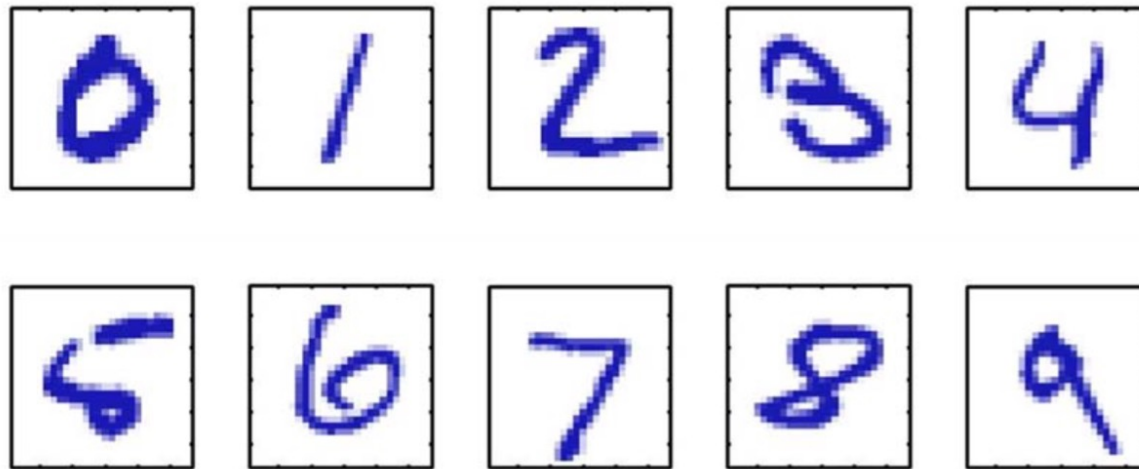
Person



Dog

Învățare supervizată

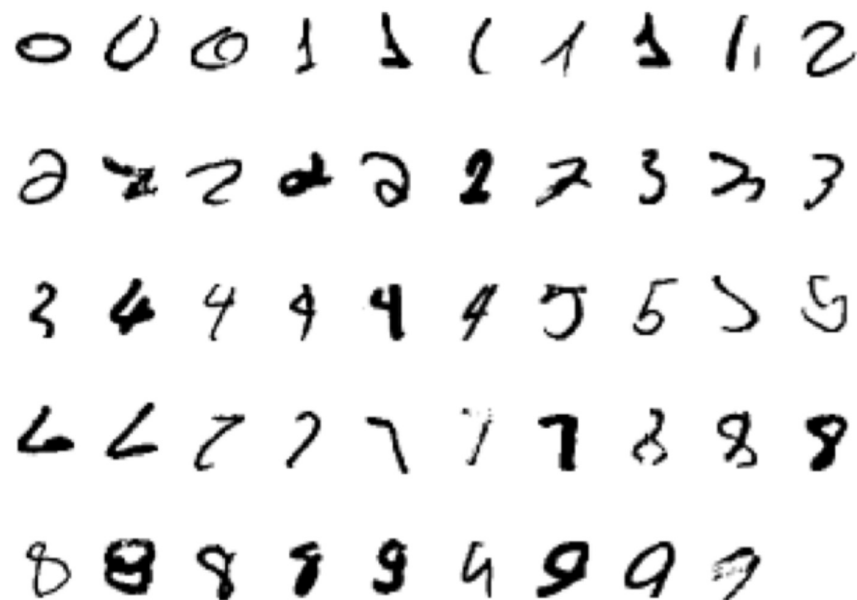
- Exemplu 2: recunoașterea caracterelor scrise de mână (setul de date MNIST)



- Imagini de 28 x 28 de pixeli
- Reprezentăm o imagine ca un vector x cu 784 de componente
- Antrenăm un clasificator $f(x)$ astfel încât:
- $f : x \rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Învățare supervizată

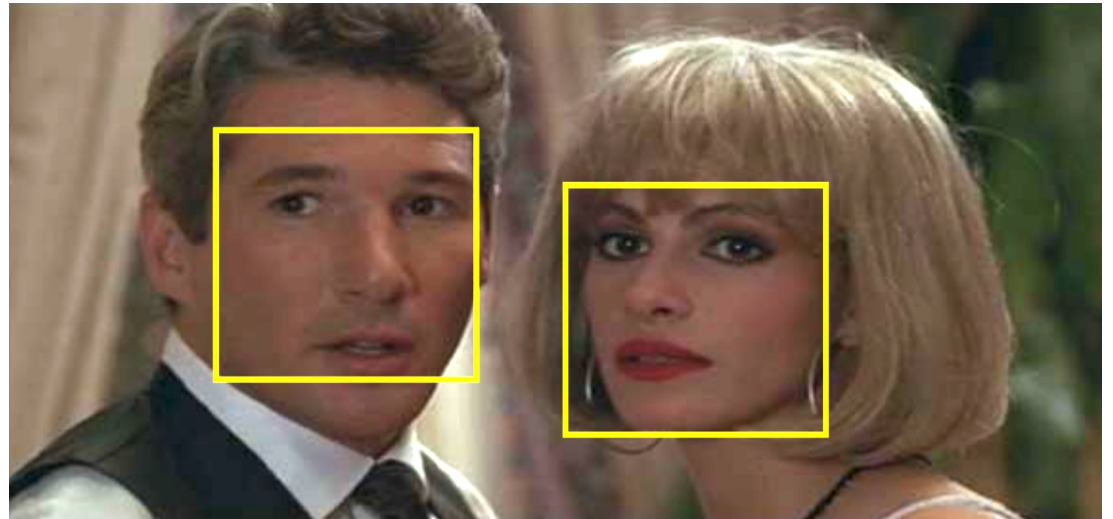
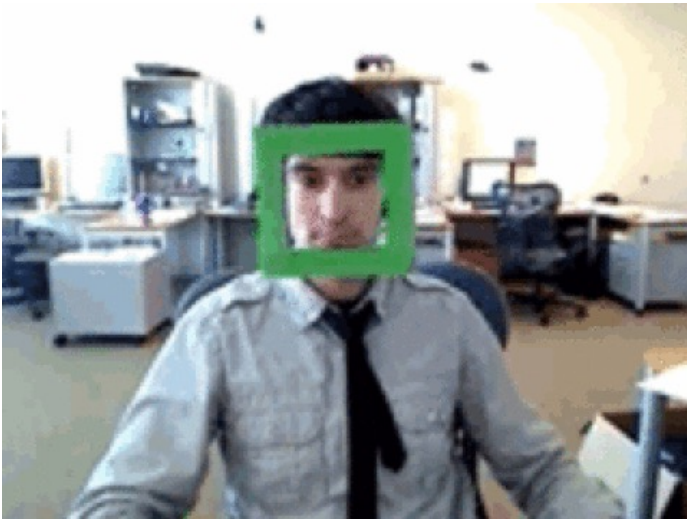
- Exemplu 2: recunoașterea caracterelor scrise de mână (setul de date MNIST)



- Pornind de la un set de antrenare, de exemplu 6000 de imagini per clasă
- Rata de eroare poate ajunge la 0.23% (cu rețele neuronale convoluționale)
- Printre primele sisteme (bazate pe învățare) comerciale utilizate pe scară largă pentru procesare de coduri poștale și cecuri bancare

Învățare supervizată

- Exemplu 3: detectare facială



- O abordare constă în plimbarea unei ferestre peste imagine
- Scopul este să clasificăm fereastra într-una din cele două clase posibile: față sau non-față (transformarea problemei într-una de clasificare)

Învățare supervizată

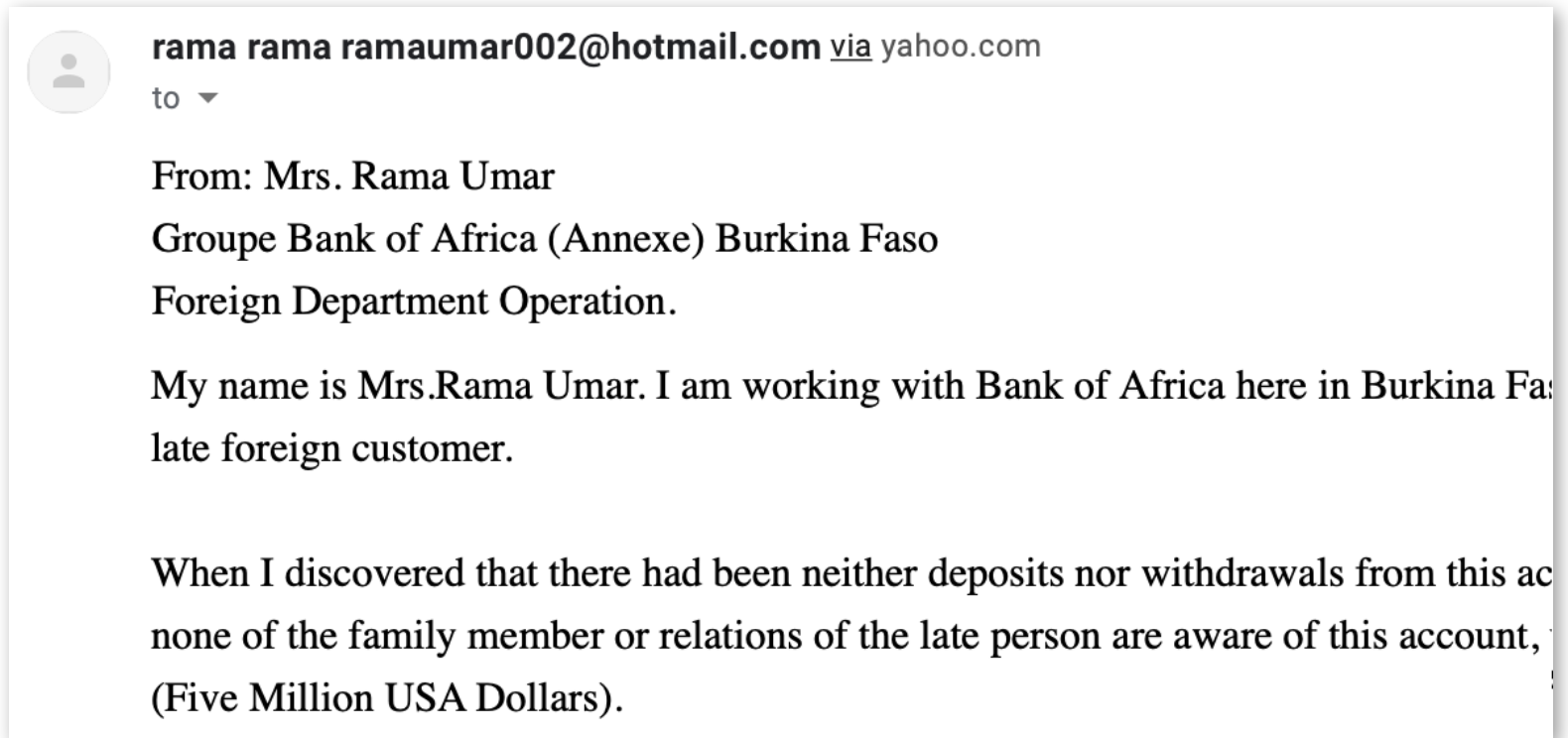
- Exemplu 3: detectare facială



- Pornim de la un set cu imagini cu fețe cu diverse variații de vârstă, gen, condiții de iluminare, dar nu translație.
- Și un set mult mai mare cu imagini care nu conțin fețe

Învățare supervizată

- Exemplu 4: detectare de spam



- Problema este de a clasifica un e-mail în spam și non-spam
- Apariția cuvântului “Dollars” este un indicator de spam
- Un exemplu de reprezentare este un vector cu frecvența cuvintelor

Aplicații de success în IA: Corectarea automată a testelor grilă

UNIVERSITATEA DIN BUCUREȘTI

  2

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ

PROBĂ DE CONCURS

Nota pe Nota pe probă după
probă contestație

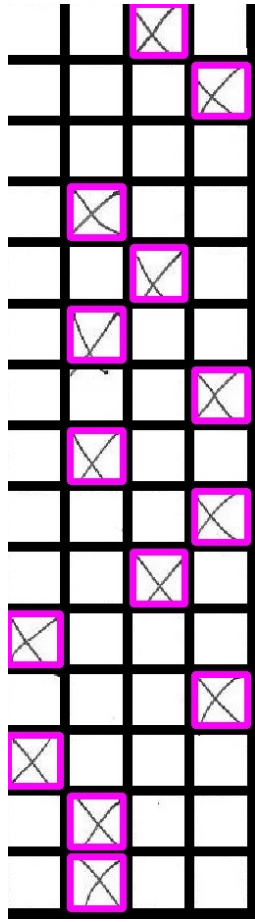


TEST GRILĂ

INFORMATICĂ ☒ FIZICĂ ☐
Număr variantă 1 Număr variantă

Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2				X
3				
4		X		
5			X	
6		X		
7				X
8		X		
9				X
10			X	
11	X			
12				X
13	X			
14		X		
15		X		

NOTĂ : Se bifează X în căsuța corespunzătoare răspunsului corect.



Numar Lucrare	Varianta	0X	1X	mX	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	Numar grile corecte	Nota
2	Informatica_1	1	14	0	C	D		B	C	B	D	B	D	C	A	D	A	B	B	2	2.2

Corectarea automată a testelor grilă

UNIVERSITATEA DIN BUCUREȘTI

2

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ

PROBĂ DE CONCURS

Nota pe probă
Nota pe probă după contestație

TEST GRILĂ

INFORMATICĂ

Număr variantă

TEST GRILĂ

INFORMATICĂ

Număr variantă

TEST GRILĂ

FIZICĂ

Număr variantă

Număr întrebare	Răspuns	A	B	C	D
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

NOTĂ : Se bifează X în căsuța corespunzătoare răspunsului corect.

NOTĂ : Se bifează X în căsuța corespunzătoare răspunsului corect.

IA pentru jocuri

- antrenate pentru a găsi cele mai bune strategii de joc
- paradigma învățării pe bază de recompensă (reinforcement learning)
 - recompensă +/- pentru câștigarea/pierderea unui joc

Şah: Deep Blue vs Kasparov

- Deep Blue (proiectat de IBM) îl învinge în 1997 pe Kasparov după ce în prealabil pierduse în 1996

- **1996: Kasparov câştigă**
“I could feel – I could smell a kind of intelligence across 1

The 1996 match

Game #	White	Black	Result	Comment
1	Deep Blue	Kasparov	1–0	
2	Kasparov	Deep Blue	1–0	
3	Deep Blue	Kasparov	½–½	Draw by mutual agreement
4	Kasparov	Deep Blue	½–½	Draw by mutual agreement
5	Deep Blue	Kasparov	0–1	Kasparov offered a draw after the 23rd move.
6	Kasparov	Deep Blue	1–0	
Result: Kasparov–Deep Blue: 4–2				

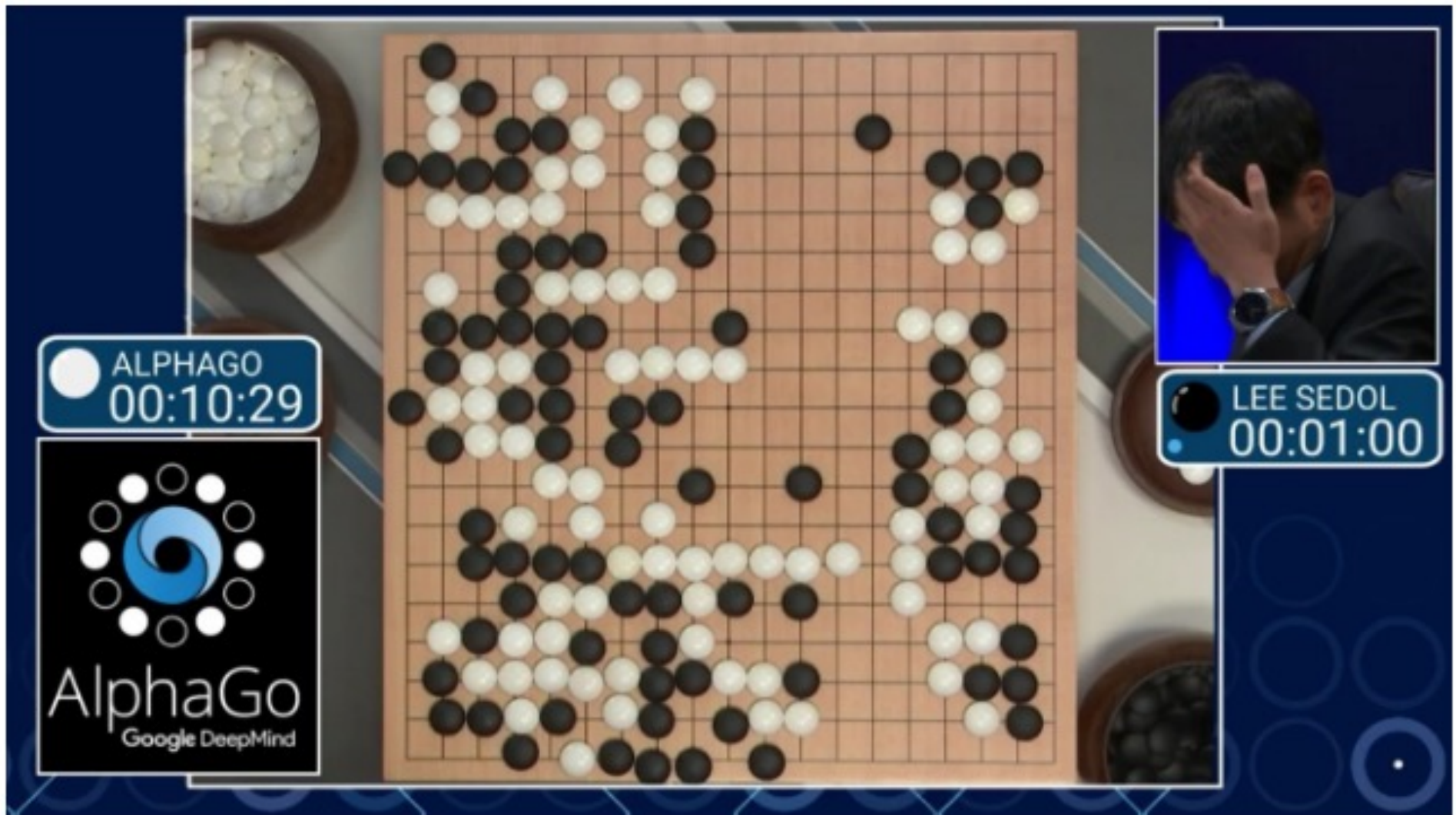
- **1997: Deep Blue câştigă**
“Deep Blue hasn't proven anything

The 1997 rematch

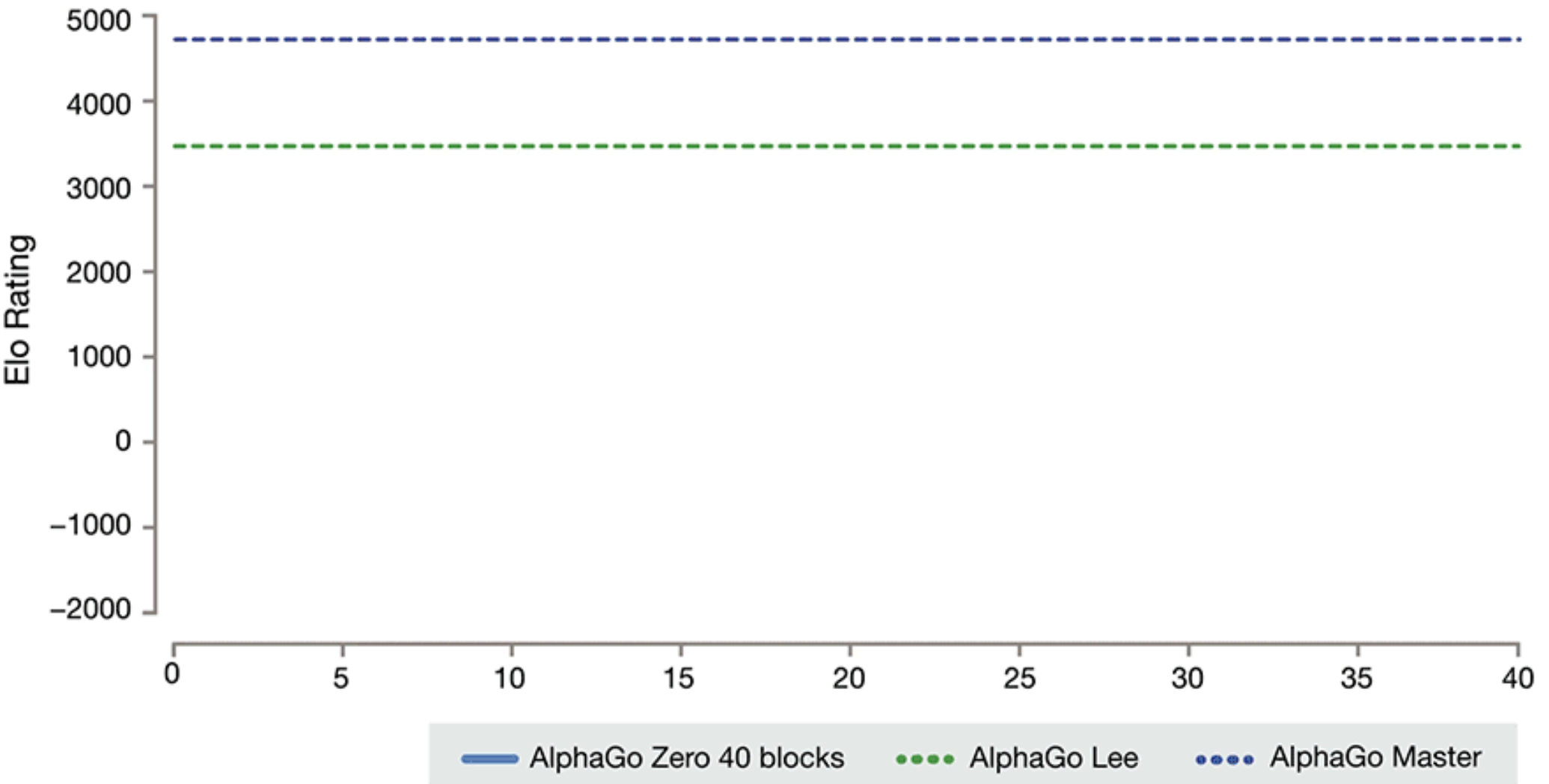
Game #	White	Black	Result	Comment
1	Kasparov	Deep Blue	1–0	
2	Deep Blue	Kasparov	1–0	
3	Kasparov	Deep Blue	½–½	Draw by mutual agreement
4	Deep Blue	Kasparov	½–½	Draw by mutual agreement
5	Kasparov	Deep Blue	½–½	Draw by mutual agreement
6	Deep Blue	Kasparov	1–0	
Result: Deep Blue–Kasparov: 3½–2½				

Go: AlphaGo vs Lee Sedol

- AlphaGo proiectat de Google DeepMind îl învinge în 2016 campionul mondial la GO

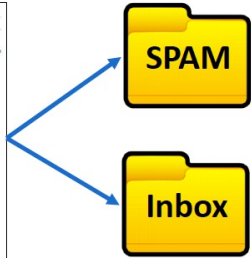
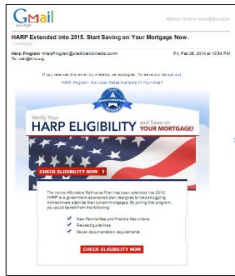


Go: AlphaGo vs AlphaGo Zero



Alte aplicații

Spam vs Inbox



Traducere automată



Recunoașterea vorbirii



Publicitate și plasare de reclame



Sisteme de recomandare

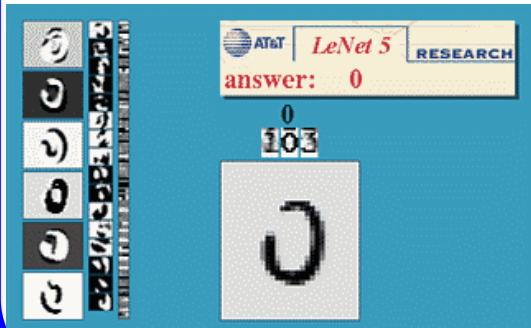


Sisteme de asistență la conducere



Alte aplicații

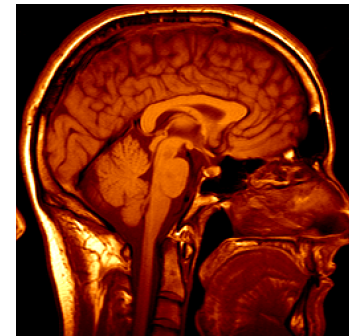
OCR



Detectare facială



Aplicații medicale



Căutare vizuală

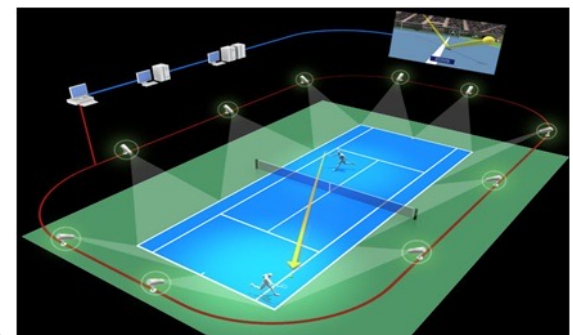


Upload an image

XBOX Kinect



Arbitraj video



Alte aplicații – generative AI



Create a template for an accountant's resume with ten years of experience. Include sections for accomplishments, skills and certifications.



[Your Name]

[Address]

[City, State, ZIP Code]

[Phone Number]

[Email Address]

Objective:

A highly skilled and detail-oriented accountant with ten years of experience in financial analysis, reporting, and taxation. Seeking a challenging position in a reputable organization where I can utilize my expertise to contribute to the financial success and growth of the company.

Summary of Qualifications:

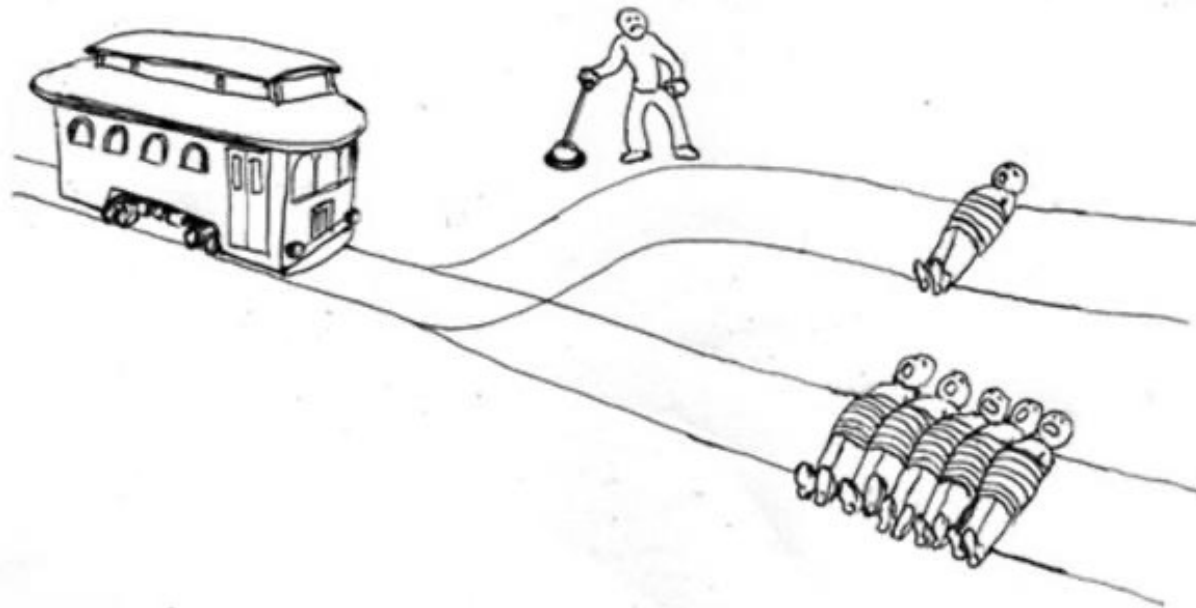
- Accomplished accountant with a proven track record of managing financial operations, optimizing processes, and delivering accurate and timely financial information.
- Expertise in financial analysis, budgeting, forecasting, and variance analysis.
- Proficient in preparing financial statements, balance sheets, income statements, and cash flow statements.

“an armchair in the shape of an avocado”



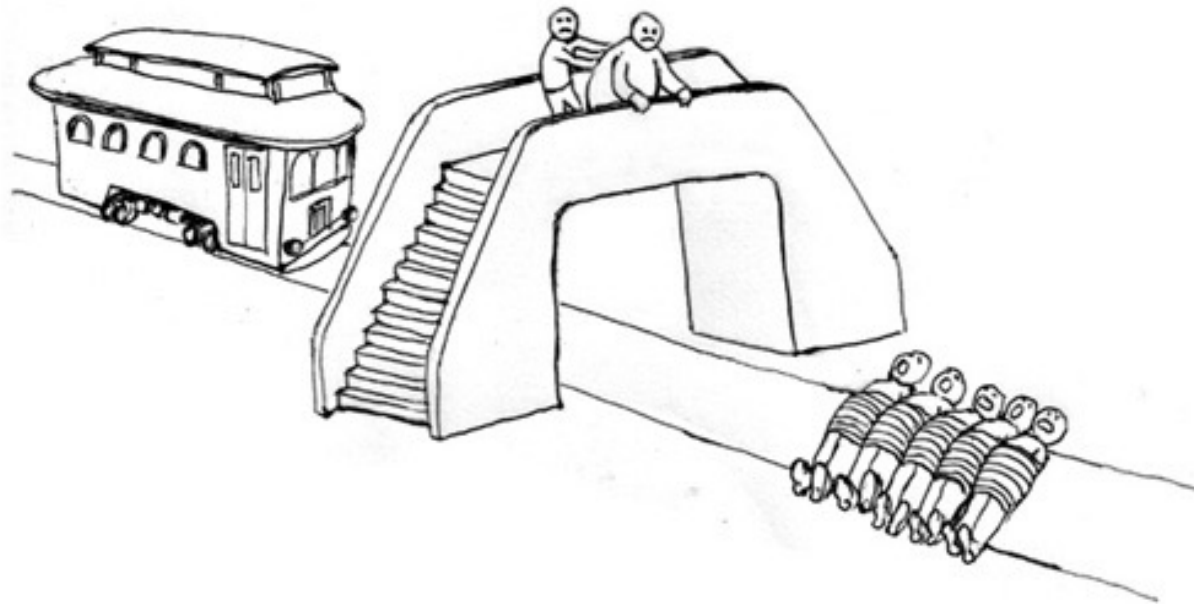
Multe aplicații interesante, dar...

- Ce este etic și ce nu?
- Trolley paradox



Multe aplicații interesante, dar...

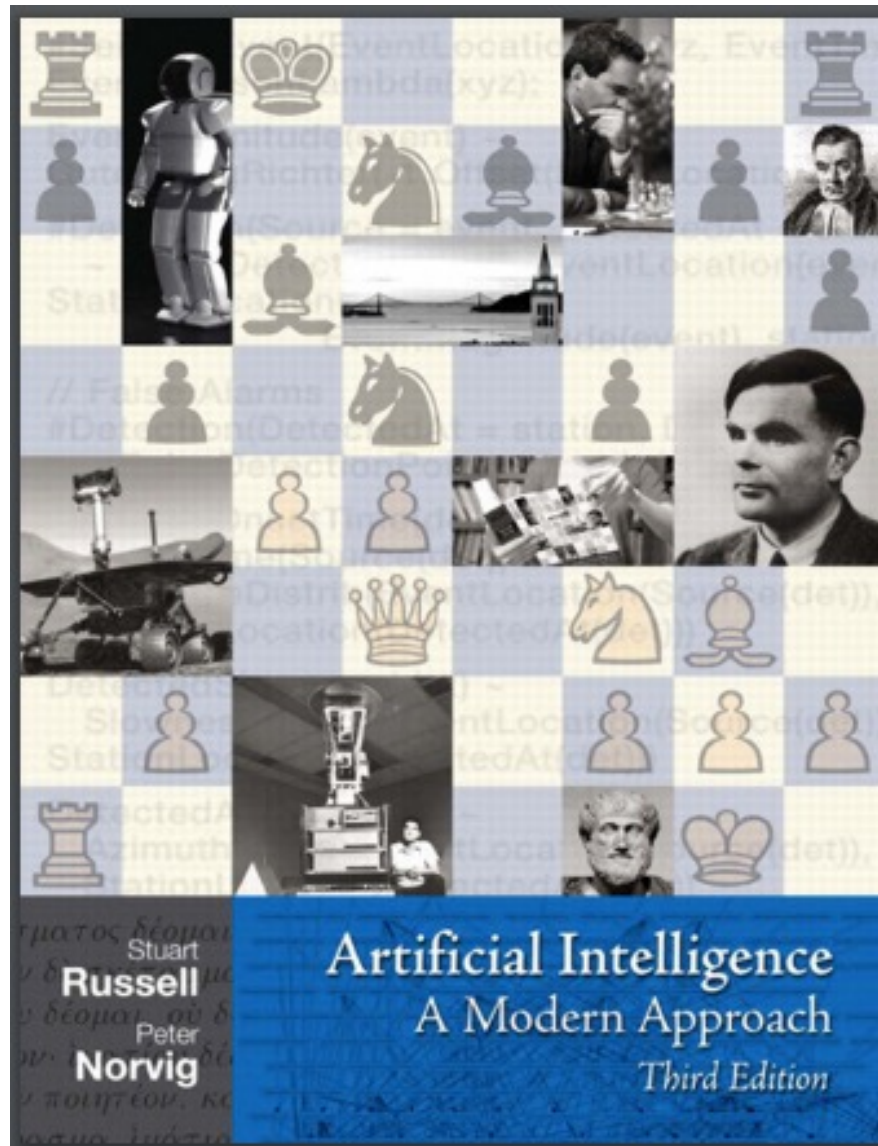
- Ce este etic și ce nu?
- Trolley paradox



Multe aplicații interesante, dar...

- Ce este etic și ce nu?
- Trolley paradox
- <http://moralmachine.mit.edu>

Bibliografie



Springer Series in Statistics

Trevor Hastie
Robert Tibshirani
Jerome Friedman

The Elements of Statistical Learning

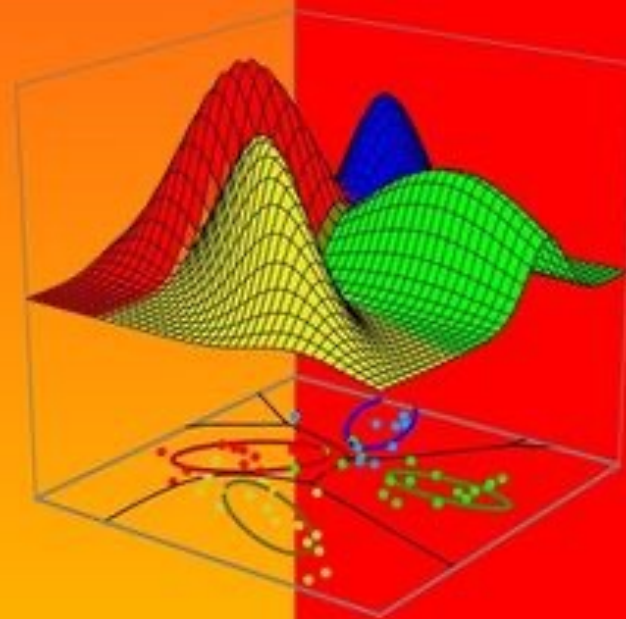
Data Mining, Inference, and Prediction

Second Edition

 Springer

Richard O. Duda
Peter E. Hart
David G. Stork

Pattern Classification

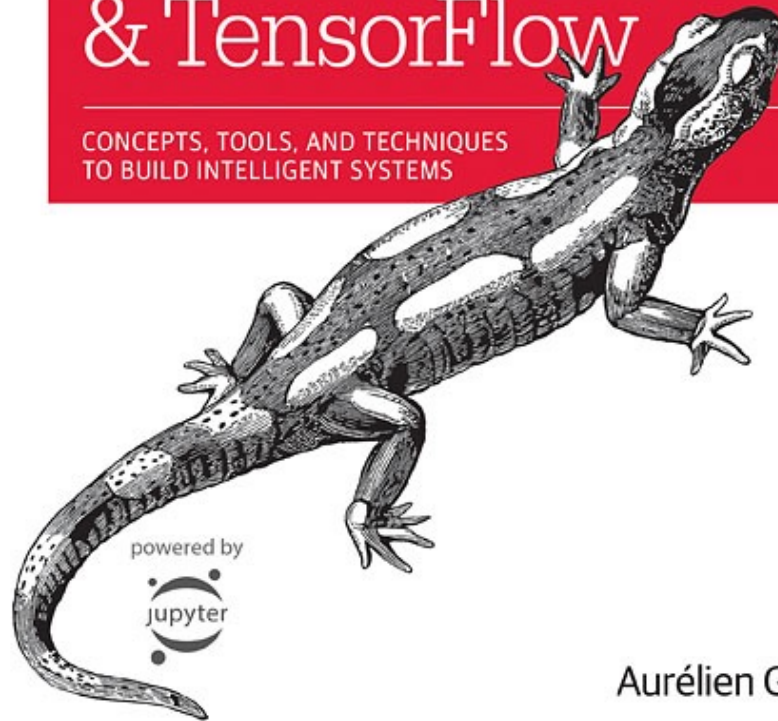


Second Edition

O'REILLY®

Hands-On Machine Learning with Scikit-Learn & TensorFlow

CONCEPTS, TOOLS, AND TECHNIQUES
TO BUILD INTELLIGENT SYSTEMS



Aurélien Géron

Advances in Computer Vision and Pattern Recognition



Radu Tudor Ionescu
Marius Popescu

Knowledge Transfer between Computer Vision and Text Mining

Similarity-based Learning Approaches

 Springer